ESCOLA POLITÉCNICA DA UNIVERSIDADE DE SÃO PAULO

DEPARTAMENTO DE ENGENHARIA DE COMPUTAÇÃO E SISTEMAS DIGITAIS

EXPERIÊNCIA 6: MODIFICAÇÃO DO SISTEMA DE SONAR

Planejamento da Experiência
Bancada A5 – Turma 1 – Prof. Edson Midorikawa

Caio Nassar de Souza – NUSP 15444601
Erwan Pierre Didelot – NUSP 18403997
Gabriel Zocal Santos – NUSP 15454039
Victor Bertelli – NUSP 15490941

PCS3645 -- LABORATÓRIO DE PROJETO DE SISTEMAS DIGITAIS II

5 de outubro de 2026

Sumário

1 Atividade 1: Projeto e Verificação do Sistema de Sonar Modificado **3**
1.1 Especificação . . . 3
1.2 Arquitetura . . . 5
1.3 Funcionamento . . . 13
1.4 Decisões de projeto e limitações . . . 14
1.5 Plano de testes . . . 14
1.6 Resultados da simulação . . . 16
1.7 Arquivo de simulação . . . 23

2 Atividade 2: Planejamento da Execução Experimental **24**
2.1 Plano de execução experimental . . . 24
2.2 Sequência de atividades na bancada . . . 30
2.3 Designação de pinos . . . 30
2.4 Compilação e arquivo QAR . . . 31

3 Atividade 3: Procedimento Experimental na Bancada **33**
3.1 Etapas intermediárias . . . 33
3.2 Sistema completo . . . 35
3.3 Ocorrências experimentais . . . 36

4 Atividade 4: Desafio **36**

1 Atividade 1: Projeto e Verificação do Sistema de Sonar Modificado

O projeto acrescenta recepção de comandos e um segundo modo ao sonar da Experiência 5. A Tabela 1 indica a cobertura do roteiro.

Tabela 1: Itens da Atividade 1 do roteiro e seções deste planejamento.

Item	Conteúdo pedido	Seção
a	Projeto do circuito, decisões de projeto e diagramas (FD e UC)	1.1 a 1.4
b	Plano de teste e *testbenches*	1.5
c	Verificação funcional no ModelSim, formas de onda anotadas	1.6
d	Arquivo `exp6_modelsim_t1ba5.zip`	1.7

1.1 Especificação

O sonar modificado opera em dois modos: localização, com varredura, e atenção, com posição fixa. A interface de `sonar2` está na Tabela 2; os requisitos M1 a M8 definem a modificação.

Tabela 2: Interface do módulo `sonar2`.

Sinal	Direção	Bits	Descrição
`clock`	Entrada	1	*Clock* de 50 MHz da placa (`CLOCK_50`).
`reset`	Entrada	1	*Reset* assíncrono, ativo em nível alto (`SW0`).
`ligar`	Entrada	1	Habilita o ciclo de operação (`SW1`); em 0, a UC retorna a `inicial`.
`entrada_serial`	Entrada	1	Comandos em 115200/7E1, recebidos do TXD do CP2102.
`echo`	Entrada	1	Pulso do HC-SR04 com largura proporcional à distância.
`sel_depuracao`	Entrada	3	Página de depuração mostrada nos displays (`SW9` a `SW7`).
`trigger`	Saída	1	Pulso de 10 µs que dispara o HC-SR04.
`pwm`	Saída	1	Controle do servomotor (período de 20 ms).
`saida_serial`	Saída	1	Mensagem `aaa,ddd#` em 115200/7E1.
`fim_posicao`	Saída	1	Pulso de 1 ciclo ao fim de cada ciclo de medida e envio, nos dois modos.

Continuação da tabela.

Sinal	Direção	Bits	Descrição
`db_modo`	Saída	1	Modo: 0 = localização; 1 = atenção (LEDR9).
`hex0` a `hex5`	Saída	7 cada	Displays HEX0 a HEX5, com a página escolhida por `sel_depuracao`.
`db_ligar`, `db_echo`, `db_trigger`, `db_pwm`, `db_saida_serial`	Saída	1 cada	Cópias dos sinais para LEDR0 a LEDR4.
`db_entrada_serial`	Saída	1	Cópia da entrada serial (LEDR5).
`db_posicao`	Saída	3	Posição do servomotor (LEDR8 a LEDR6).

O modo de localização mantém os requisitos da Experiência 5: habilitação por `ligar` (R1), ciclo nominal de 2 s (R2), pulso de `fim_posicao` (R3), mensagem `aaa,ddd#` em 115200/7E1 (R4), oito posições com retorno de 111 a 000 (R5), reutilização dos componentes (R6), divisão em FD e UC (R7) e *timeout* de 10 s (R8).

M1 Depois do `reset`, o sonar começa no modo de localização (`db_modo` = 0).

M2 Ao receber `'a'`, passa ao modo de atenção: interrompe a movimentação do servomotor e continua medindo e transmitindo a cada 2 s.

M3 Ao receber `'v'`, volta ao modo de localização, com movimentação, medidas e transmissão a partir da posição em que estava.

M4 Só `'a'` e `'v'`, minúsculos, são comandos. Qualquer outro caractere é ignorado.

M5 Os comandos chegam pela `entrada_serial`, em 115200/7E1, e são recebidos pelo `rx_serial_7E1` fornecido, como componente interno do FD.

M6 A saída `db_modo` indica o modo; os sinais de depuração da Experiência 5 são mantidos ou adaptados.

M7 Restrição de projeto: o novo modo é implementado alterando a máquina de estados da UC, com uma sequência de estados para cada modo, sem blocos lógicos sobre sinais de controle no FD.

M8 Multiplexação dos recursos de depuração conforme a Seção 1.2-C do roteiro; neste projeto, aplicada aos displays.

Três pontos que o roteiro não fixa foram decididos pelo grupo (Seção 1.4): **D1** religar (`ligar` de 0 para 1) também volta ao modo de localização, como o `reset`; **D2** um caractere com erro de paridade é descartado, mesmo que seus 7 bits formem `'a'` ou `'v'`; **D3** `fim_posicao` pulsa no fim de cada ciclo nos dois modos.

1.2 Arquitetura

O módulo `sonar2` interliga a UC, o FD e o painel de depuração (Tabela 3). Os parâmetros `ESPERA` e `TIMEOUT` definem a pausa de 2 s e o limite de 10 s da medida; `CHAR_ATENCAO` e `CHAR_VOLTAR` definem os comandos.

Tabela 3: Módulos do projeto.

Módulo	Função	Origem
`sonar2`	Módulo principal: UC, FD, displays e LEDs	Esta experiência
`sonar2_uc`	UC: FSM de sequência (12 estados) e FSM de modo	`sonar_uc` da Exp. 5, modificado
`sonar2_fd`	FD: o da Exp. 5 mais a recepção de comandos	`sonar_fd` da Exp. 5, modificado
`rx_serial_7E1`	Recepção serial 115200/7E1	Fornecido, sem alteração
`tx_serial_7E1`	Transmissão serial 115200/7E1	Exp. 5
`interface_hcsr04`	Disparo do sensor e medida em cm	Exp. 5
`controle_servo_8`	PWM do servomotor, 8 posições	Exp. 5
`circuito_pwm`	Gerador PWM de 8 larguras	Exp. 5
`rom_angulos_8x24`	Ângulo de cada posição em ASCII	Exp. 5 (roteiro)
`mux_8x1_n`	Multiplexador 8×1 de N bits (mensagem e páginas)	Exp. 5
`contador_cm`, `gerador_pulso`, `contador_bcd_3digitos`	Componentes do `interface_hcsr04`	Exp. 5
`contador_m`, `registrador_n`, `deslocador_n`, `hexa7seg`	Componentes de biblioteca	Biblioteca (os mesmos do pacote do receptor)

Os módulos com FD e UC próprios incluem os arquivos `_fd` e `_uc` (por exemplo, `rx_serial_7E1_fd` e `rx_serial_uc`). Os demais módulos da Exp. 5 foram reaproveitados sem alteração.

1.2.1 Recepção serial (`rx_serial_7E1`)

O receptor fornecido é utilizado sem alteração (Figura 1). Sua implementação contém:

- **Temporização:** um `contador_m` com $M = 434$ (50 MHz / 115200) é reiniciado ao detectar o *start bit*. A saída `meio`, com $Q = 216$, gera a primeira amostra próxima ao centro desse bit; as seguintes ocorrem a cada 434 ciclos.
- **Armazenamento:** um `deslocador_n` de 11 bits, carregado com 7FFH no início do quadro, recebe RX pela esquerda. Um contador de módulo 11 conta as amostras e um `registrador_n` de 8 bits guarda {paridade, dado}.
- **Controle:** em `inicial`, a UC aguarda RX = 0. O estado `preparacao` reinicia a temporização e carrega o deslocador; `espera` e `recepcao` se alternam a cada *tick*.

Após a décima amostra, `registrar` guarda o dado e `final_rx` emite `pronto` por um ciclo.

- **Saídas:** `dados_ascii` contém os sete bits de dado e `paridade` contém o bit recebido. A saída `paridade_par` é o XNOR de redução de {paridade, dado}; vale 1 quando a paridade par está correta. O dado permanece disponível até o quadro seguinte.

Na simulação, `pronto` ocorre 82,56 µs após o início do *start bit* (9,5 bits mais alguns ciclos), antes do fim do *stop bit*. O receptor retorna a `inicial` a tempo de receber um quadro consecutivo, sem repouso adicional; esse caso é verificado pelo `rx_serial_7E1_tb`.

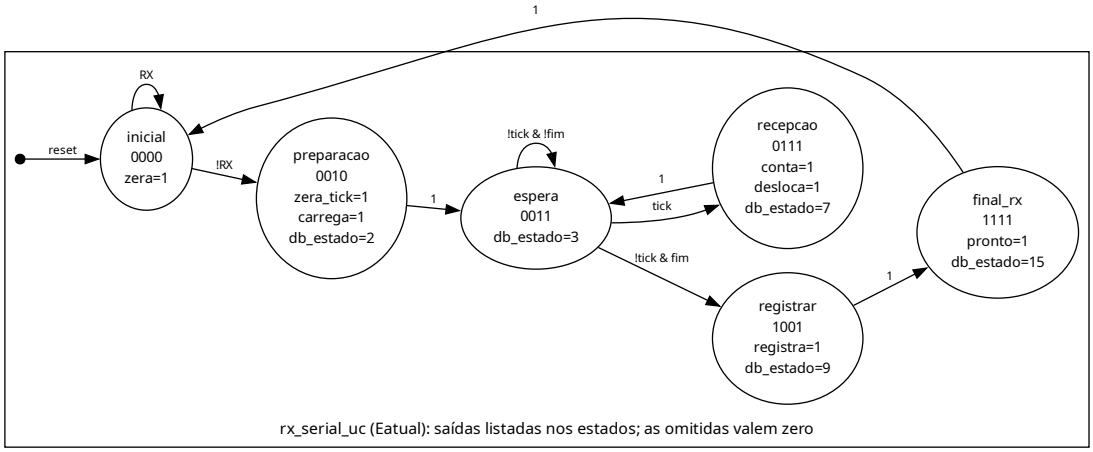


Figura 1: UC do `rx_serial_7E1`, extraída do RTL pelo Yosys (`make figura-fsm MOD=rx_serial_uc SAIDAS=1`).

1.2.2 Fluxo de dados

O FD da Figura 2 reutiliza a estrutura da Experiência 5 e acrescenta o receptor U10 e os comparadores de comandos, em laranja. As entradas superiores são controles da UC; as saídas inferiores são condições devolvidas à UC. Os sinais de reinício foram omitidos do desenho.

- **Sensor (U1):** `interface_hcsr04` recebe `mede`, dispara o sensor e registra a distância em três dígitos BCD. Seu `pronto` é `fim_medida`. O reinício ocorre por `reset`, `zera` ou `limpa_sensor`, este último após o *timeout*.
- **Transmissor (U2):** `tx_serial_7E1` recebe `transmite` e envia o caractere selecionado pelo multiplexador. Seu `pronto` é `fim_transmissao`.
- **PWM (U3):** `controle_servo_8` gera o pulso correspondente à posição atual (Tabela 4).
- **Posição (U4):** um `contador_m` de módulo 8 guarda a posição, avança com `conta_posicao` e retorna de 111 a 000.

- **Mensagem (U5 a U7):** `rom_angulos_8x24` fornece o ângulo em ASCII; um contador de módulo 8 seleciona o caractere e `mux_8x1_n`, com $N = 7$, o encaminha ao TX. O sinal `fim_char` indica o oitavo caractere (`#`).
- **Espera (U8):** `contador_m` com $M =$ `ESPERA` conta durante a pausa anterior à medida.
- **Timeout (U9):** `contador_m` com $M =$ `TIMEOUT` limita a espera pela medida a 10 s. É zerado apenas por `mede`: a UC passa por `dispara_medida` antes de contar em `espera_medida`, dispensando o reinício por `zera` da Experiência 5.
- **Receptor (U10):** `rx_serial_7E1` recebe `entrada_serial` e é reiniciado apenas pelo `reset` global, permanecendo ativo com o sonar desligado.
- **Comparadores:** `pronto`, paridade correta e igualdade com o caractere configurado geram um pulso de comando de um ciclo. Os comparadores atuam sobre o dado recebido; a interpretação do comando cabe à UC.

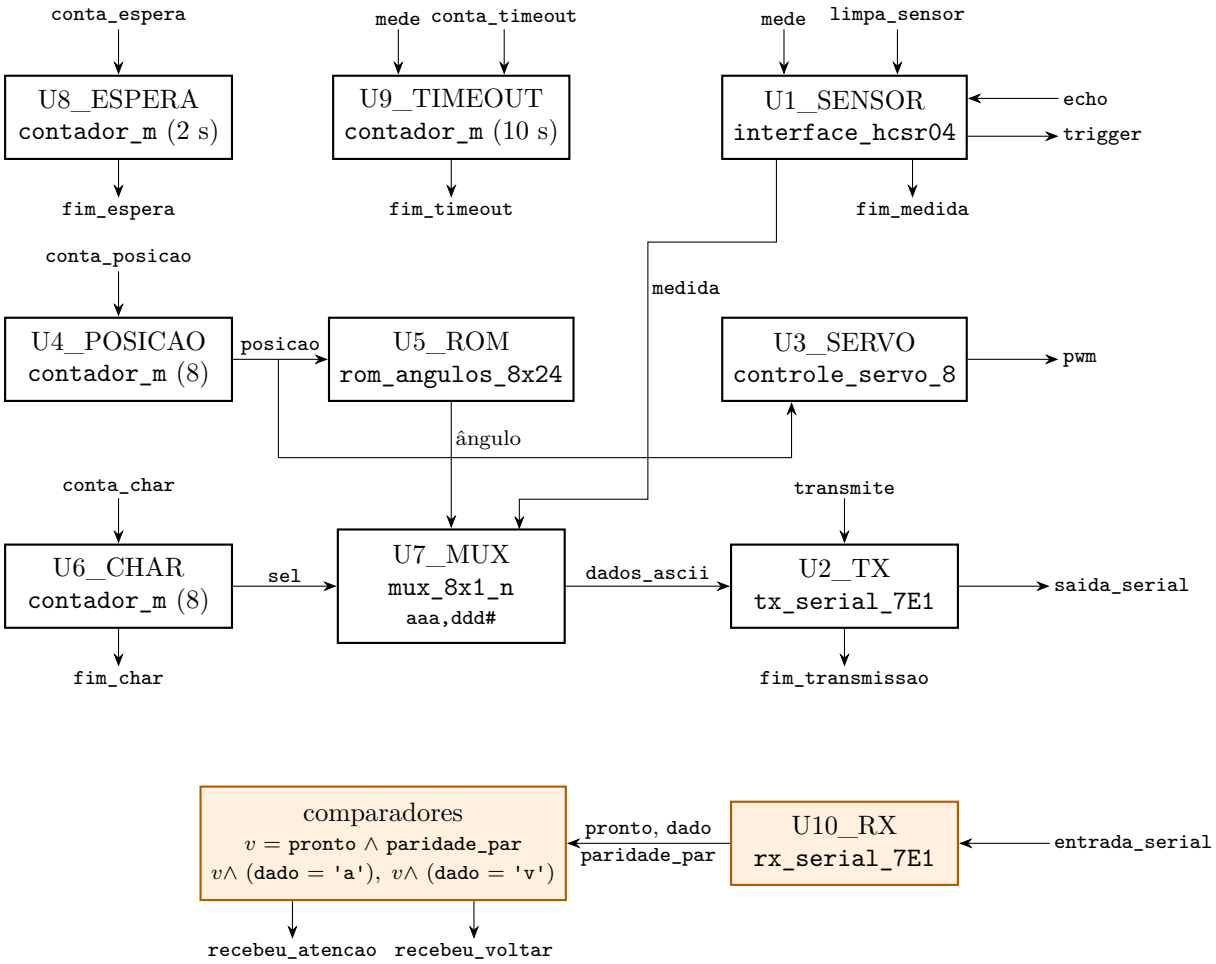


Figura 2: Diagrama de blocos do FD do sonar modificado. Em laranja, a recepção de comandos acrescentada nesta experiência.

Tabela 4: Posições do servomotor (`controle_servo_8`, sem alteração desde a Experiência 5).

`posicao`	Largura do pulso	Ciclos de *clock*	Ângulo
000	0,7 ms	35 000	20°
001	0,914 ms	45 700	40°
010	1,129 ms	56 450	60°
011	1,343 ms	67 150	80°
100	1,557 ms	77 850	100°
101	1,771 ms	88 550	120°
110	1,986 ms	99 300	140°
111	2,2 ms	110 000	160°

Tabela 5: Sinais entre UC e FD.

Sinal	Sentido	Efeito
`zera`	UC → FD	Zera sensor, transmissor, posição (000), caracteres e espera.
`conta_espera`	UC → FD	Incrementa o temporizador de 2 s.
`mede`	UC → FD	Pulso de 1 ciclo que inicia uma medida e zera o temporizador de 10 s.
`conta_timeout`	UC → FD	Incrementa o temporizador de 10 s.
`limpa_sensor`	UC → FD	Zera só o `interface_hcsr04`, após o *timeout*.
`transmite`	UC → FD	Pulso de 1 ciclo que inicia o envio de um caractere.
`conta_char`	UC → FD	Avança para o próximo caractere da mensagem.
`conta_posicao`	UC → FD	Avança o servomotor para a próxima posição (só no modo de localização).
`fim_espera`	FD → UC	Passaram-se 2 s.
`fim_medida`	FD → UC	A medida terminou e está no registrador.
`fim_timeout`	FD → UC	Passaram-se 10 s sem `fim_medida`.
`fim_transmissao`	FD → UC	O quadro serial do caractere atual terminou.
`fim_char`	FD → UC	O caractere selecionado é o último (`#`).
`recebeu_atencao`	FD → UC	**Novo.** Pulso: chegou `'a'` com paridade correta.
`recebeu_voltar`	FD → UC	**Novo.** Pulso: chegou `'v'` com paridade correta.

As interfaces de controle e condição estão na Tabela 5. A Listagem 1 mostra as alterações do FD em relação à Experiência 5: `-` indica linhas removidas e `+`, acrescentadas. Os arquivos-fonte completos acompanham o arquivo de simulação (Seção 1.7).

```
-module sonar_fd #(
+module sonar2_fd #(
     parameter       ESPERA       = 100_000_000, // 2s @ 50MHz
     parameter       TIMEOUT      = 500_000_000, // 10s sem resposta do sensor
+    parameter [7:0] CHAR_ATENCAO = "a",         // comando que entra no modo atencao
+    parameter [7:0] CHAR_VOLTAR  = "v"          // comando que volta ao modo normal
 ) (
     ...
+    input  wire        entrada_serial,
+    output wire        recebeu_atencao, // pulso: chegou CHAR_ATENCAO valido
+    output wire        recebeu_voltar,  // pulso: chegou CHAR_VOLTAR valido
 ...
     ) U9_TIMEOUT (
-        .zera_s (zera | mede  ),
+        .zera_s (mede         ),
 ...
+    rx_serial_7E1 U10_RX (
+        .clock       (clock           ),
+        .reset       (reset           ),
+        .RX          (entrada_serial  ),
+        .pronto      (s_rx_pronto     ),
+        .dados_ascii (s_rx_dado       ),
+        .paridade_par(s_rx_paridade_ok),
+        ...
+    );
+    assign s_rx_valido     = s_rx_pronto & s_rx_paridade_ok;
+    assign recebeu_atencao = s_rx_valido & (s_rx_dado == CHAR_ATENCAO[6:0]);
+    assign recebeu_voltar  = s_rx_valido & (s_rx_dado == CHAR_VOLTAR[6:0]);
```

Listagem 1: Alterações do FD em relação à Experiência 5.

1.2.3 Unidade de controle

A UC mantém duas máquinas de estados:

- **Modo (`Matual`, 1 bit):** recebe os pulsos de comandos durante a espera, a medida ou o envio. Em `inicial`, retorna à localização; `db_modo` expõe o valor registrado.
- **Sequência (`Eatual`, 4 bits):** executa espera, medida e transmissão. Após o último caractere, bifurca conforme o modo: localização passa por `muda_posicao` e `final_posicao`; atenção passa diretamente por `final_atencao`.

Os dois ramos geram `fim_posicao`. A ausência de `conta_posicao` no ramo de atenção mantém o servomotor na mesma posição, com o PWM ativo, atendendo à restrição M7. Os diagramas e as Tabelas 6 e 7 detalham as transições.

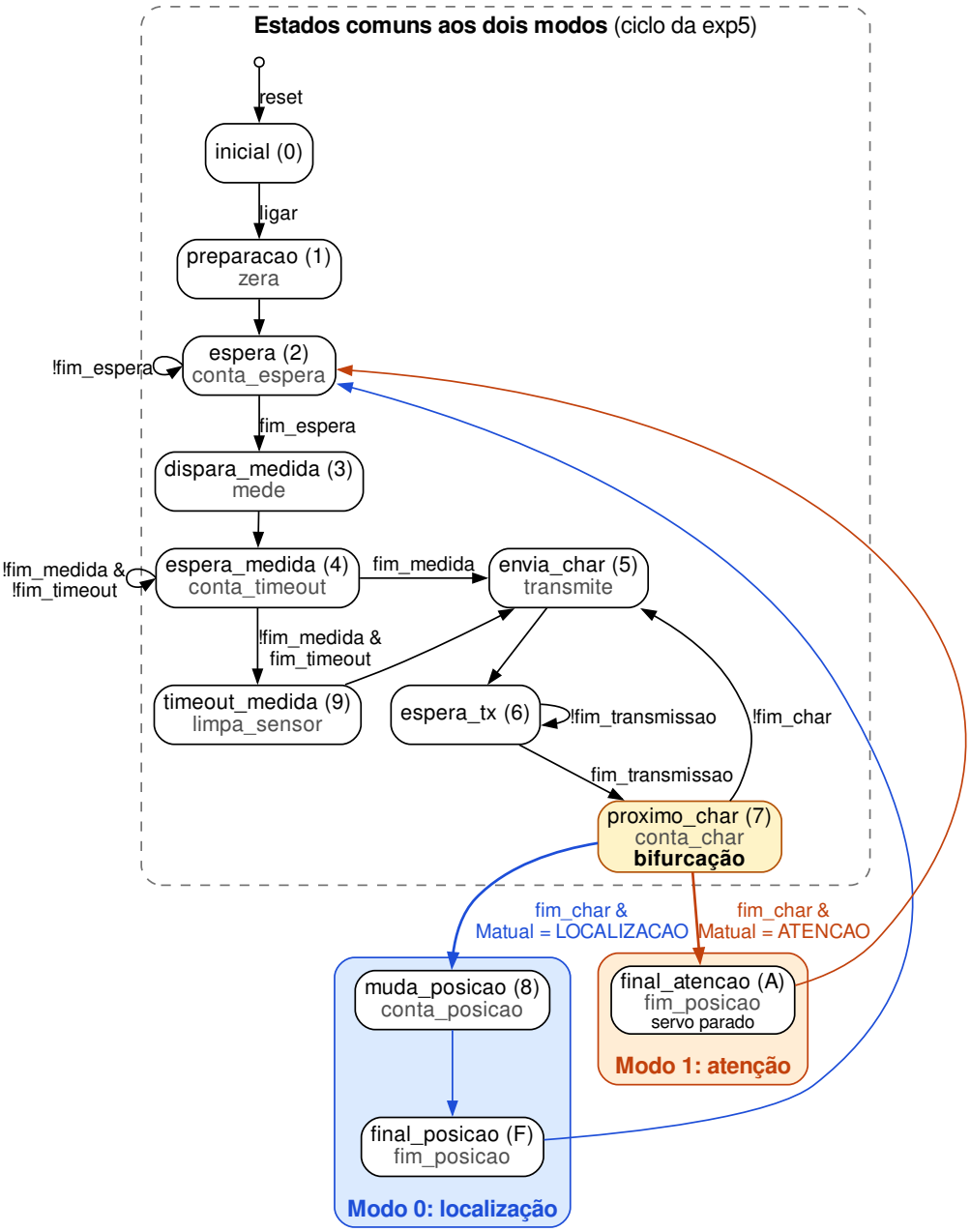


Figura 3: Diagrama de transição de estados da FSM de sequência (`Eatual`). As transições supõem `ligar` = 1 (com `ligar` = 0, qualquer estado vai para `inicial`); as saídas de Moore aparecem abaixo do nome, e o número entre parênteses é o código do estado mostrado no display.

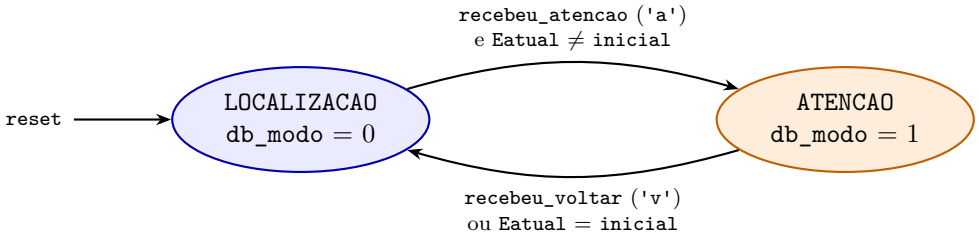


Figura 4: FSM de modo da UC (`Matual`, 1 bit), descrita a partir do RTL (o extrator do Yosys não detecta FSMs de 1 bit). Os comandos são aceitos ao longo do ciclo; em `inicial`, o modo permanece em localização.

Tabela 6: Estados da FSM de sequência (`Eatual`). O código é o dígito mostrado no display do estado da UC.

Estado (código)	Grupo	Saída ativa	Condição	Próximo estado
`inicial` (0)	comum	—	`ligar` = 1	`preparacao`
`preparacao` (1)	comum	`zera`	—	`espera`
`espera` (2)	comum	`conta_espera`	`fim_espera` = 1	`dispara_medida`
`dispara_medida` (3)	comum	`mede`	—	`espera_medida`
`espera_medida` (4)	comum	`conta_timeout`	`fim_medida` = 1	`envia_char`
			`fim_timeout` = 1	`timeout_medida`
`timeout_medida` (9)	comum	`limpa_sensor`	—	`envia_char`
`envia_char` (5)	comum	`transmite`	—	`espera_tx`
`espera_tx` (6)	comum	—	`fim_transmissao` = 1	`proximo_char`
`proximo_char` (7)	comum	`conta_char`	`fim_char` = 0	`envia_char`
			`fim_char` = 1, modo 0	`muda_posicao`
			`fim_char` = 1, modo 1	`final_atencao`
`muda_posicao` (8)	modo 0	`conta_posicao`	—	`final_posicao`
`final_posicao` (F)	modo 0	`fim_posicao`	—	`espera`
`final_atencao` (A)	modo 1	`fim_posicao`	—	`espera`

Nos estados com espera (2, 4 e 6), a UC permanece no estado enquanto a condição não ocorre. Com `ligar` = 0, vai para `inicial` a partir de qualquer estado.

Tabela 7: FSM de modo (`Matual`).

Modo atual (`db_modo`)	Condição	Próximo modo
`LOCALIZACAO` (0)	`recebeu_atencao` = 1 e `Eatual` ≠ `inicial`	`ATENCAO`
`ATENCAO` (1)	`recebeu_voltar` = 1	`LOCALIZACAO`
qualquer	`Eatual` = `inicial` (desligado, `reset` ou religar)	`LOCALIZACAO`

O modo atua na seleção do próximo estado da UC. As portas de reinício dos submódulos no FD são independentes do modo. As Listagens 2 e 3 mostram a implementação.

```
// FSM de modo: proximo modo
always @(*) begin
    if (Eatual == inicial)
        Mprox = LOCALIZACAO;    // desligado: reset/religar -> modo normal
    else
        case (Matual)
            LOCALIZACAO: Mprox = recebeu_atencao ? ATENCAO     : LOCALIZACAO;
            ATENCAO:     Mprox = recebeu_voltar  ? LOCALIZACAO : ATENCAO;
            default:     Mprox = LOCALIZACAO;
        endcase
end
```

Listagem 2: FSM de modo na `sonar2_uc`.

```
     input  wire      ligar,
+    input  wire      recebeu_atencao, // pulso: chegou o comando 'a'
+    input  wire      recebeu_voltar,  // pulso: chegou o comando 'v'
     ...
+    output wire      db_modo,         // 0 = localizacao, 1 = atencao
 ...
+    parameter final_atencao  = 4'hA; // modo 1: fim do ciclo, servo parado
+    parameter LOCALIZACAO    = 1'b0; // modo 0 (normal, da exp5)
+    parameter ATENCAO        = 1'b1; // modo 1 (servo parado)
 ...
-    proximo_char:  Eprox = fim_char ? muda_posicao : envia_char;
+    proximo_char:  Eprox = !fim_char           ? envia_char    :
+                           (Matual == ATENCAO) ? final_atencao : muda_posicao;
     muda_posicao:  Eprox = final_posicao;      // modo 0
     final_posicao: Eprox = espera;             // modo 0
+    final_atencao: Eprox = espera;             // modo 1
 ...
-    fim_posicao   = (Eatual == final_posicao);
+    fim_posicao   = (Eatual == final_posicao) || (Eatual == final_atencao);
```

Listagem 3: Alterações da FSM de sequência em relação à Experiência 5.

1.2.4 Depuração: páginas dos displays e LEDs

Um multiplexador 8×1 de 24 bits seleciona a página dos seis displays (Tabela 8). As chaves `SW9`, `SW8` e `SW7` escolhem uma de oito páginas: quatro de depuração e quatro livres. Os LEDs mantêm atribuições fixas para permitir observar modo e posição em qualquer página (Tabela 9).

Tabela 8: Páginas dos displays (`sel_depuracao` = `SW9 SW8 SW7`).

Página	HEX5	HEX4	HEX3	HEX2	HEX1	HEX0
000 sonar	estado da UC	estado HC-SR04	posição	distância (3 dígitos BCD)		
001 serial	estado TX	caractere TX (ASCII)		estado RX	caractere RX (ASCII)	
010 mensagem	ângulo (3 dígitos BCD)			distância (3 dígitos BCD)		
011 comandos	estado da UC	modo	índice do caractere	paridade ok	caractere RX (ASCII)	
1xx livres	000000					

Caracteres em ASCII hexadecimal: `'a'` = 61, `'v'` = 76, `'0'` = 30, `','` = 2C, `'#'` = 23. A página 010 mostra o ângulo da posição atual do servomotor. "Paridade ok" vale 1 quando o último quadro recebido tinha a paridade par correta. Códigos de estado: UC do sonar na Tabela 6; HC-SR04: 0 inicial, 1 preparação, 2 envia *trigger*, 3 espera *echo*, 4 medida, 5 armazenamento, F final; TX: 0 inicial, 1 preparação, 3 espera, 7 transmissão, F final; RX: Figura 1.

A Listagem 4 mostra a montagem das páginas. As entradas 100 a 111 ficam reservadas para novos sinais de depuração.

```
// cada pagina: {HEX5, HEX4, HEX3, HEX2, HEX1, HEX0}, 4 bits por display
assign s_pagina0 = {s_estado, s_estado_sensor, d_posicao, s_medida};     // 000
assign s_pagina1 = {s_estado_tx, d_dado_tx, s_estado_rx, d_dado_rx};     // 001
assign s_pagina2 = {s_angulo, s_medida};                                 // 010
assign s_pagina3 = {s_estado, d_modo, d_char, d_paridade_ok, d_dado_rx}; // 011
assign s_pagina4 = 24'h000000;                                           // 100 livre
```

Listagem 4: Páginas dos displays no `sonar2` (as livres vão até `s_pagina7`).

Tabela 9: LEDs de depuração.

LED	Sinal	Utilidade
LEDR0	`db_ligar`	Nível da chave `ligar`.
LEDR1	`db_echo`	Nível de `echo`; medir sua duração no analisador.
LEDR2	`db_trigger`	Pulso de `trigger` (10 µs, praticamente invisível).
LEDR3	`db_pwm`	PWM do servomotor (brilho fraco e constante).
LEDR4	`db_saida_serial`	Nível da saída serial; aceso em repouso.
LEDR5	`db_entrada_serial`	Nível da entrada serial; aceso em repouso.
LEDR8 a LEDR6	`db_posicao`	Posição do servomotor em binário: parada no modo de atenção.
LEDR9	`db_modo`	Aceso no modo de atenção.

1.3 Funcionamento

Como exemplo, com posição 011 (80°) e distância de 64 cm, o sonar transmite `080,064#`. Se recebe `'a'` antes da bifurcação, encerra esse ciclo em `final_atencao`; os ciclos seguintes

mantêm o ângulo e atualizam a distância. Com `'v'`, o fim do ciclo volta ao ramo de reposicionamento e avança para 100 (100°).

A Tabela 10 reúne os tempos do circuito. A pausa de 2 s é acrescida do tempo de medida, transmissão e transições da UC; portanto, o intervalo entre disparos depende da duração do `echo`.

Tabela 10: Temporização (*clock* de 50 MHz).

Grandeza	Ciclos de *clock*	Tempo
Pausa antes da medida (`ESPERA`)	100 000 000	2 s
Timeout da medida (`TIMEOUT`)	500 000 000	10 s
Período do PWM do servomotor	1 000 000	20 ms
Largura do `trigger`	500	10 µs
Largura de `echo` por centímetro	2941	≈ 58,8 µs
Duração de um bit serial	434	≈ 8,68 µs
Quadro 7E1 recebido (10 bits)	4340	86,8 µs
Start bit até o `pronto` do receptor	4128	82,56 µs
`pronto` até a troca de modo	1	20 ns
Mensagem de 8 caracteres	≈ 38 200	≈ 0,76 ms

1.4 Decisões de projeto e limitações

- **Modo independente:** registrar o modo permite receber comandos durante todo o ciclo, sem duplicar os estados comuns aos dois modos. A bifurcação mantém cada medida e mensagem em uma única posição.
- **Condição inicial (D1):** o *reset* e o religamento retornam à posição 000 e ao modo de localização. Durante `inicial`, os comandos não alteram o modo.
- **Validade dos dados (D2):** quadros com paridade incorreta são rejeitados antes da interpretação do comando.
- **Sincronismo dos testes (D3):** emitir `fim_posicao` em ambos os modos permite usar o mesmo marcador de término de ciclo na simulação e na bancada.

Limitações. O modo só é consultado na bifurcação do fim do ciclo: um comando que chega depois dela só tem efeito na bifurcação seguinte. As entradas assíncronas não possuem sincronizadores. Desligar retorna a UC a `inicial`, mas o PWM permanece ativo e os submódulos já acionados podem terminar sua atividade. O pulso de `fim_posicao` dura 20 ns e só é visível no analisador lógico; os pulsos `recebeu_*` são internos e, na placa, só se observam pelo efeito em `db_modo`.

1.5 Plano de testes

A verificação tem duas etapas: simulação no ModelSim (CT1 a CT20, abaixo) e testes na bancada (Seção 2.1.3).

1.5.1 Procedimento de simulação

O `simula_todos.do` compila os arquivos-fonte no modo SystemVerilog (`vlog -sv`) e executa 13 *testbenches*: sete da biblioteca, quatro dos componentes reutilizados, um do receptor e um do sistema completo. A cobertura está na Tabela 13.

O teste do receptor retoma os oito quadros fornecidos e acrescenta `'a'`, `'v'` e dois quadros consecutivos. Confere dados, paridade e um pulso de `pronto` por quadro. Foi reescrito com verificações automáticas e sem o `struct` do exemplo fornecido.

1.5.2 Elaboração do *testbench* do sonar

O `sonar2_tb` emula o computador e o sensor. Usa `ESPERA` = 10 000 ciclos (200 µs) e `TIMEOUT` = 1 000 000 ciclos (20 ms).

O computador emulado envia quadros 7E1 e recebe os oito caracteres da mensagem, verificando dado, paridade e *stop bit*. O sensor responde 400 µs após o `trigger`, com `echo` de duração proporcional à distância. A cada `fim_posicao`, o teste compara mensagem, modo e posição. Processos paralelos permitem aplicar estímulos durante a medida e a transmissão.

São 20 casos (Tabela 11) e 63 verificações. O *watchdog* interrompe uma execução travada após 300 ms; como não incrementa `erros`, sua mensagem deve ser conferida no transcrito. O teste CT7 compara o tempo até o fim do ciclo, incluindo a transmissão; a duração do *timeout* é medida separadamente na Figura 10.

Tabela 11: Casos de teste de simulação do `sonar2_tb`.

Caso	Condição inicial e estímulo	Resultado esperado
CT1 a CT3	Localização; posições 000, 001 e 010; objetos a 5, 17 e 35 cm.	`020,005#`; `040,017#`; `060,035#`. Avanço até 011.
CT4	Posição 011; `'a'` 20 µs após o ciclo anterior; 64 cm.	Modo 1 após o quadro; `080,064#`; posição 011.
CT5	Atenção; posição 011; 30 cm.	`080,030#`; posição mantida.
CT6	Atenção; `'A'`, `'x'` e `'V'` com paridade correta; `'v'` com paridade errada. Enviar durante a medida; 9 cm.	Modo 1 após cada quadro; `080,009#`; posição 011.
CT7	Atenção; posição 011; ausência de `echo`.	`080,000#` após o *timeout*; posição 011.
CT8	Posição 011; `'v'` durante a espera; 45 cm.	Modo 0; `080,045#`; avanço para 100.
CT9	Localização; posição 100; 100 cm.	`100,100#`; avanço para 101.
CT10	Posição 101; `'a'` a partir do primeiro *start bit* da mensagem; 12 cm.	Modo 1 antes da bifurcação; `120,012#`; posição 101.
CT11	Atenção; `'a'` repetido; 20 cm.	Modo 1; `120,020#`; posição mantida.

Continuação da tabela.

Caso	Condição inicial e estímulo	Resultado esperado
CT12	`'v'` seguido de `'a'`; posição 101; 25 cm.	Modo 1; `120,025#`; posição mantida.
CT13	`'a'` seguido de `'v'`; posição 101; 33 cm.	Modo 0; `120,033#`; avanço para 110.
CT14 a CT16	Localização; posições 110, 111 e 000; 44, 52 e 61 cm.	`140,044#`; `160,052#`; `020,061#`. Retorno de 111 a 000.
CT17	Posição 001; `'a'`; 77 cm.	Modo 1; `040,077#`; posição 001.
CT18	Atenção; desligar por 100 µs e religar; 72 cm.	Desligado: modo 0 e estado 0. Religado: `020,072#`; avanço para 001.
CT19	Atenção; *reset* de 2 µs; 50 cm.	Modo 0; `020,050#`; avanço para 001.
CT20	`'x'`; páginas 000, 001, 010, 011, 100 e 111.	Respectivamente: `201050`, `030078`, `040050`, `200178`, `000000`, `000000`.

Sensibilidade dos testes. Quatro alterações incorretas foram injetadas, uma por vez, em uma cópia do RTL. Todas foram detectadas pelo `sonar2_tb` (Tabela 12).

Tabela 12: Erros injetados no RTL e casos que os detectam.

Erro injetado	Detectado em	Erros apontados
Sem bifurcação: `proximo_char` sempre vai a `muda_posicao`	CT4 em diante	26
Sem conferir a paridade (`valido` = `pronto`)	CT6 em diante	24
Modo mantido ao religar (sem forçar `LOCALIZACAO`)	CT18	2
Comando por nível (`dado` = `'a'`) em vez de pulso	CT18	1

1.6 Resultados da simulação

A Tabela 13 resume os resultados registrados no ModelSim (Questa). O transcrito está na Listagem 5; as formas de onda anotadas e as medidas da Tabela 14 documentam os eventos observados. Estes resultados são de simulação; a validação física será registrada na Atividade 3.

Tabela 13: Cobertura e resultados da simulação.

Grupo (*testbenches*)	Cobertura	Resultado
Biblioteca (7)	Decodificador, registrador, deslocador, contadores e gerador de pulso.	OK
Componentes reutilizados (4)	ROM de ângulos, PWM do servomotor, interface HC-SR04 e TX serial.	OK
Receptor serial (1)	11 casos: dados, paridade, pulso de pronto e quadros consecutivos.	OK
Sistema completo (1)	CT1 a CT20; 63 verificações de mensagem, modo, posição e displays.	OK

```
# Caso 4: posicao 3, objeto a 64 cm, servo para -- t=7549690
#   envia 'a' -- t=7569690
#   OK: modo atencao
#   OK: "080,064#"
#   OK: servo na posicao 3
...
# Fim da simulacao: todos os casos OK
# ==================== resumo ====================
# hexa7seg_tb               OK
# registrador_n_tb          OK
# deslocador_n_tb           OK
# contador_m_tb             OK
# contador_bcd_3digitos_tb  OK
# gerador_pulso_tb          OK
# contador_cm_tb            OK
# rom_angulos_8x24_tb       OK
# controle_servo_8_tb       OK
# interface_hcsr04_tb       OK
# tx_serial_7E1_tb          OK
# rx_serial_7E1_tb          OK
# sonar2_tb                 OK
```

Listagem 5: Trecho do transcrito do ModelSim para o `simula_todos.do`.

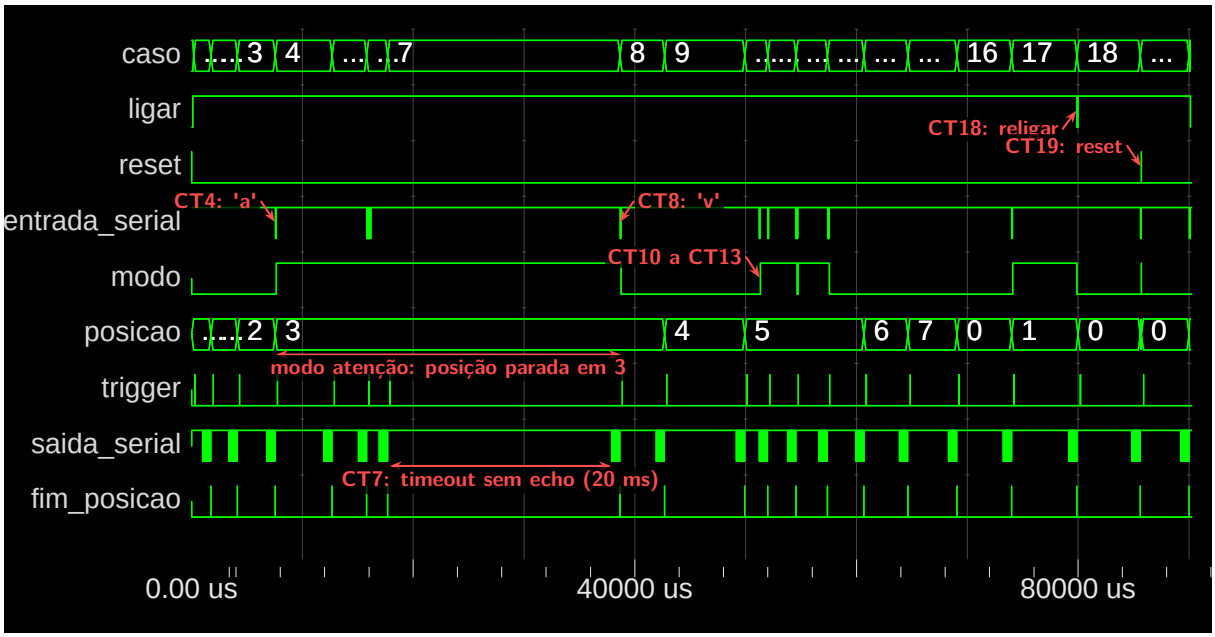


Figura 5: Visão geral do `sonar2_tb` (CT1 a CT20, 90 ms): enquanto `modo` = 1, a posição do servomotor não muda.

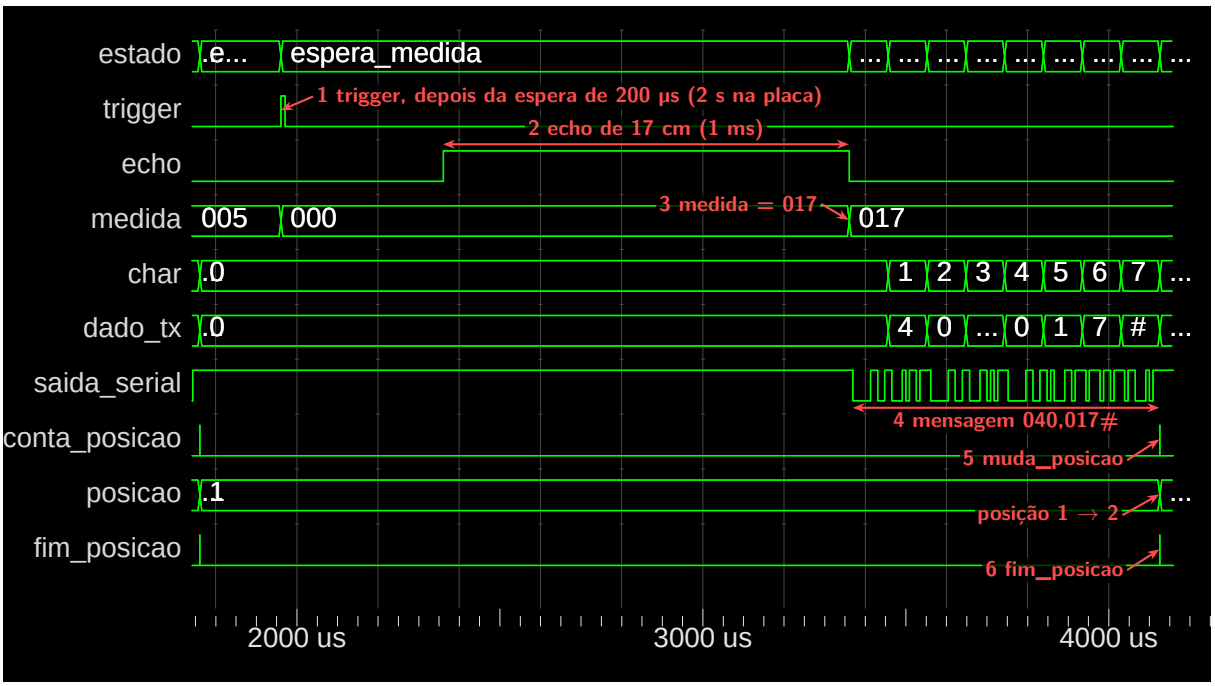


Figura 6: Um ciclo no modo de localização (CT2): espera, `trigger`, *echo* de 17 cm, mensagem `040,017#`, mudança de posição e `fim_posicao`.

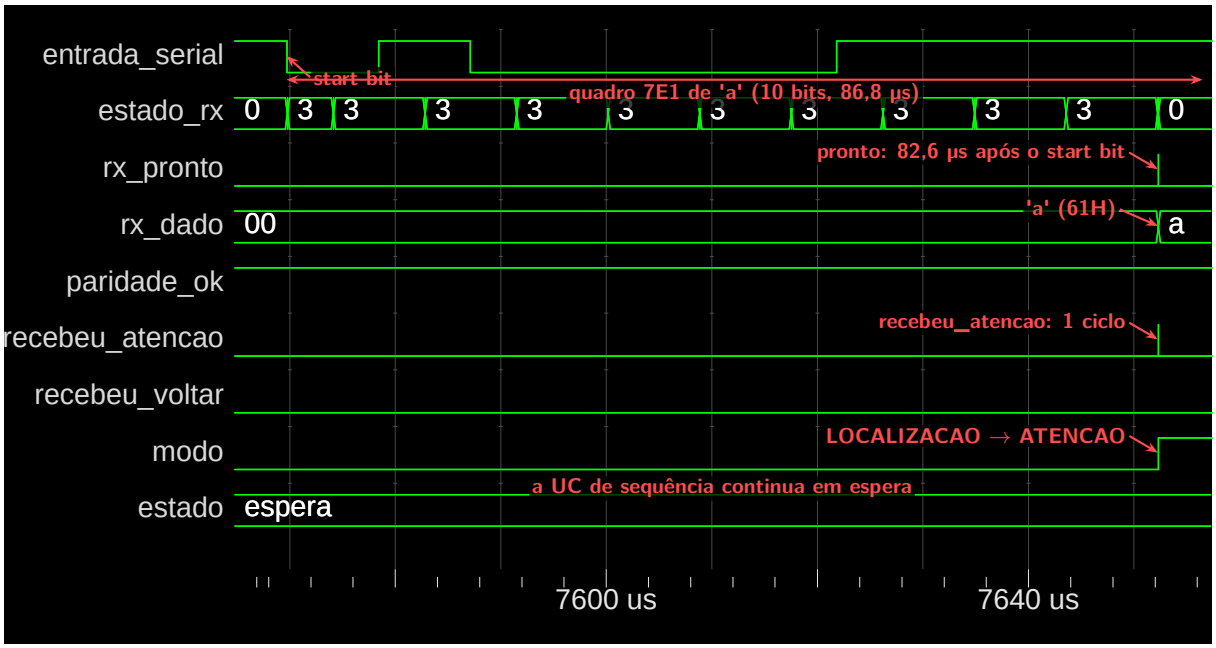


Figura 7: Recepção de `'a'` durante a espera (CT4) e troca para o modo de atenção.

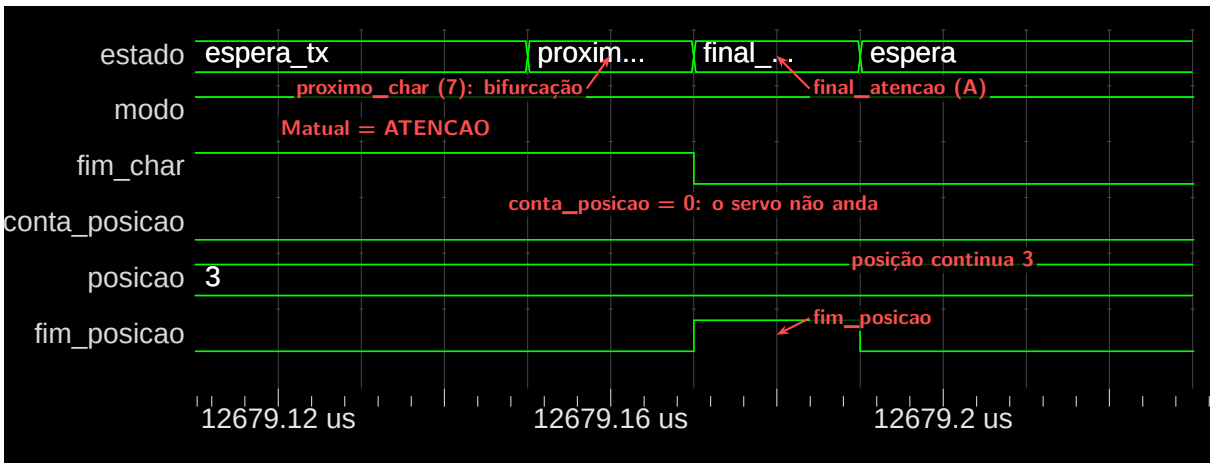


Figura 8: Fim do ciclo do CT4 no modo de atenção: a bifurcação leva a `final_atencao` e a posição não muda.

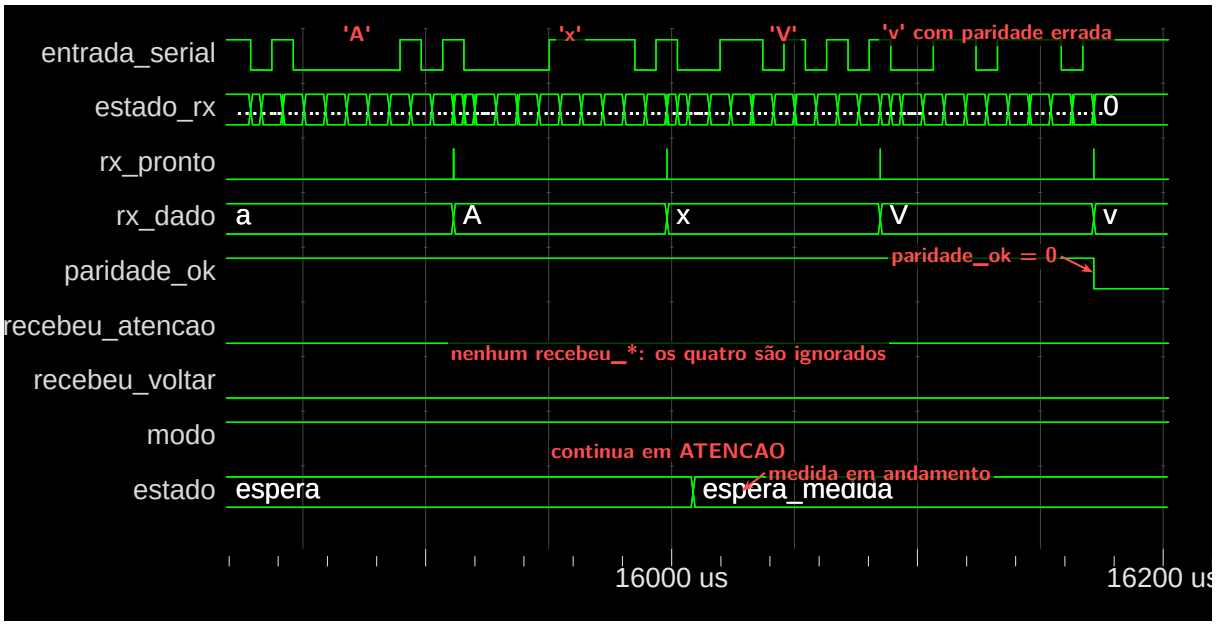


Figura 9: Caracteres ignorados (CT6): nenhum gera `recebeu_*`, inclusive o `'v'` com erro de paridade.

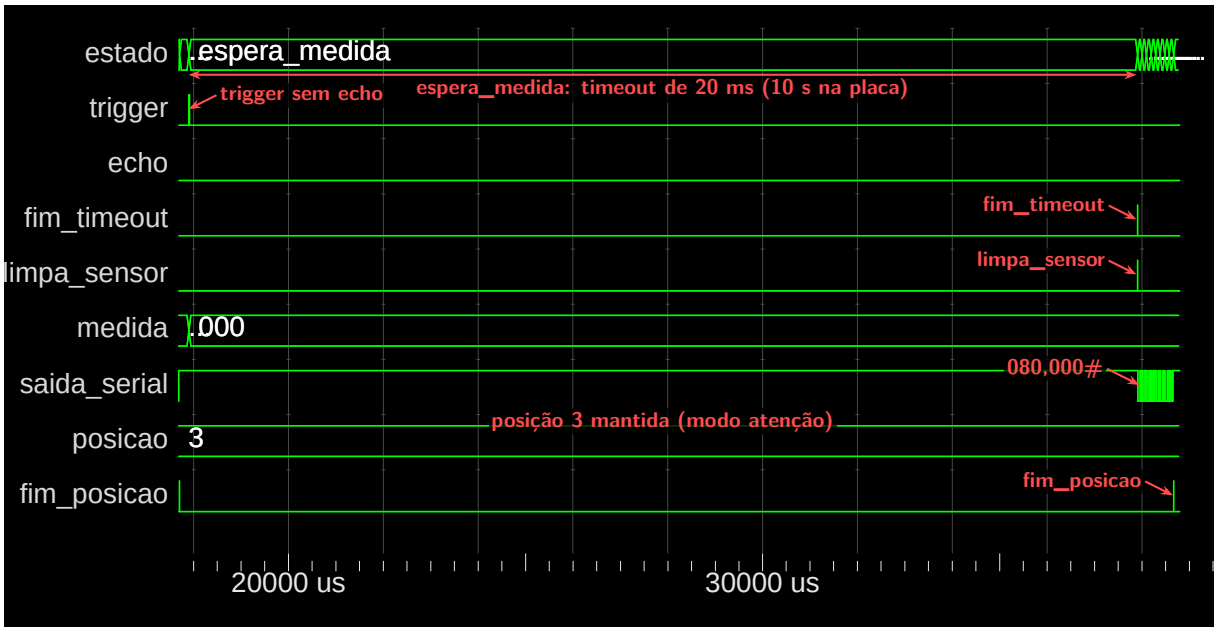


Figura 10: Medida sem *echo* no modo de atenção (CT7): *timeout*, `080,000#` e posição mantida.

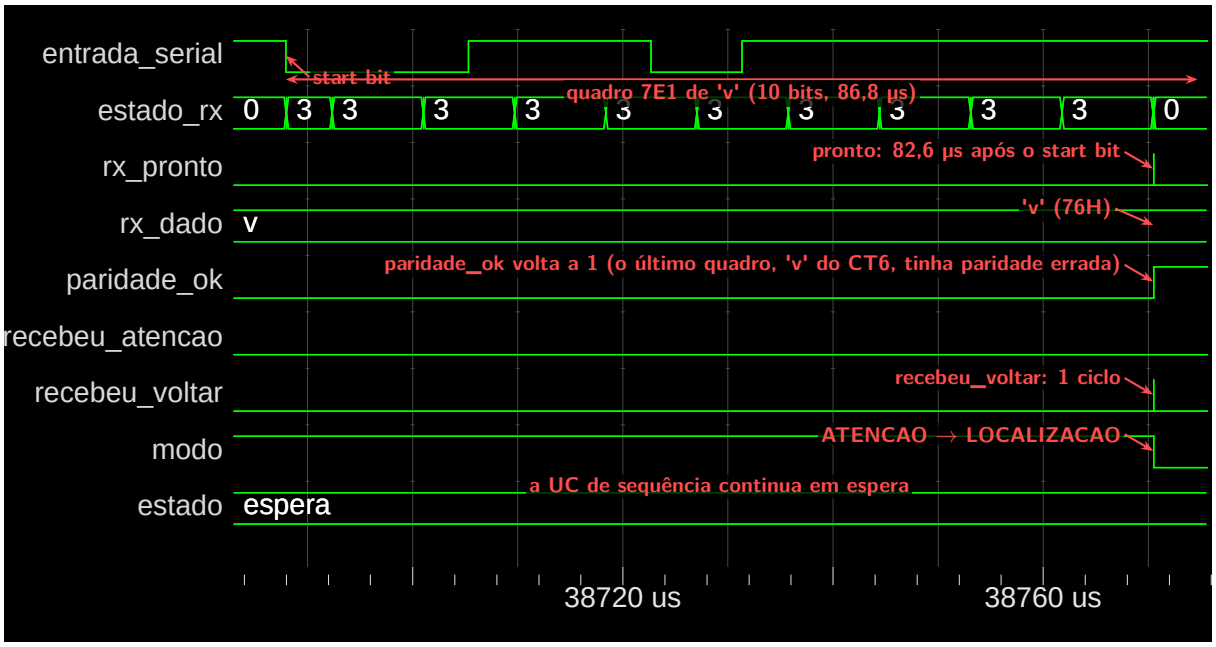


Figura 11: Recepção de `'v'` (CT8) e volta ao modo de localização.

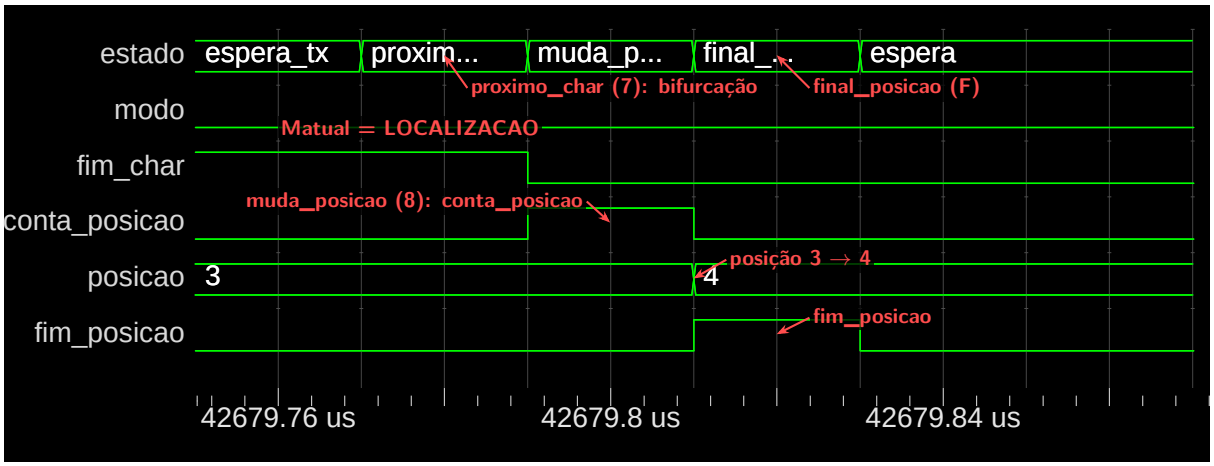


Figura 12: Fim do ciclo do CT8 no modo de localização: `muda_posicao` avança o servomotor de 011 para 100, a partir da posição em que estava.

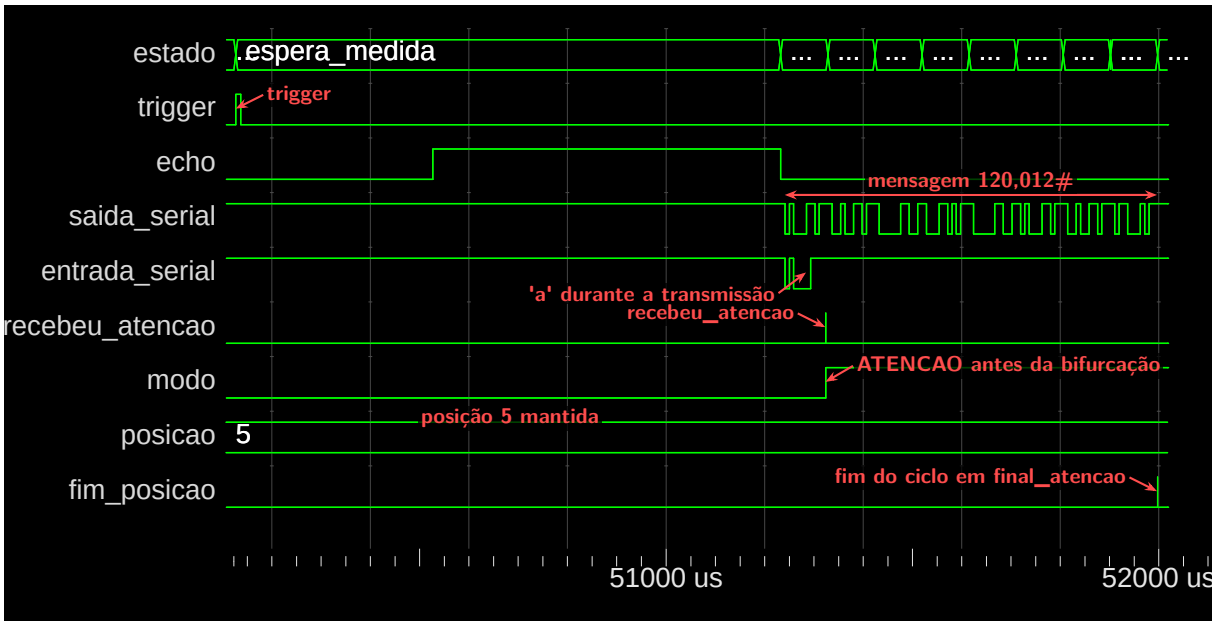


Figura 13: Comando `'a'` recebido durante a transmissão da mensagem (CT10): vale no mesmo ciclo.

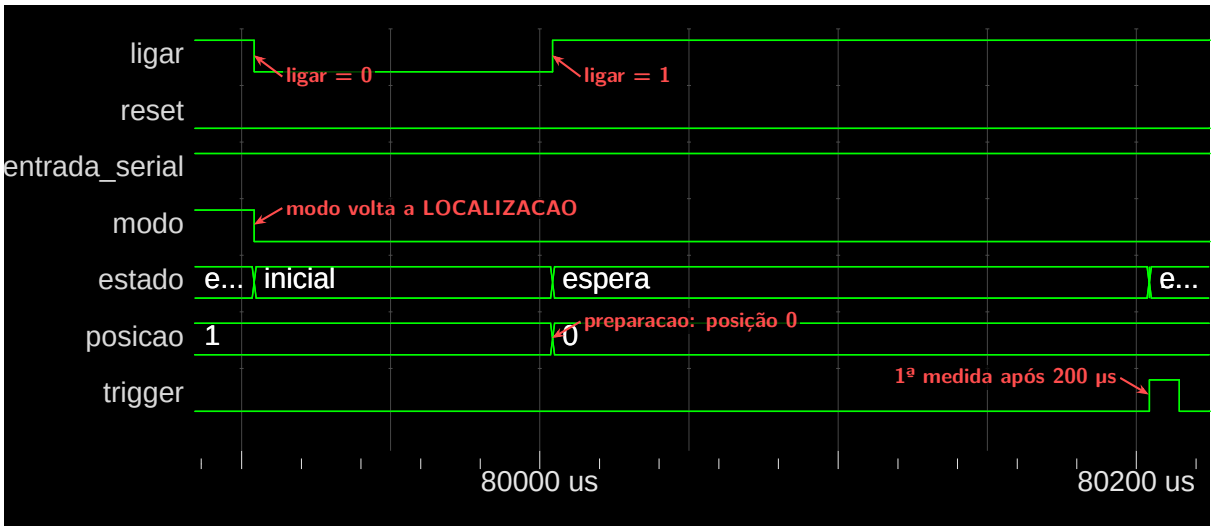


Figura 14: Religar no modo de atenção (CT18): o sonar desligado fica no modo de localização e recomeça na posição 000.

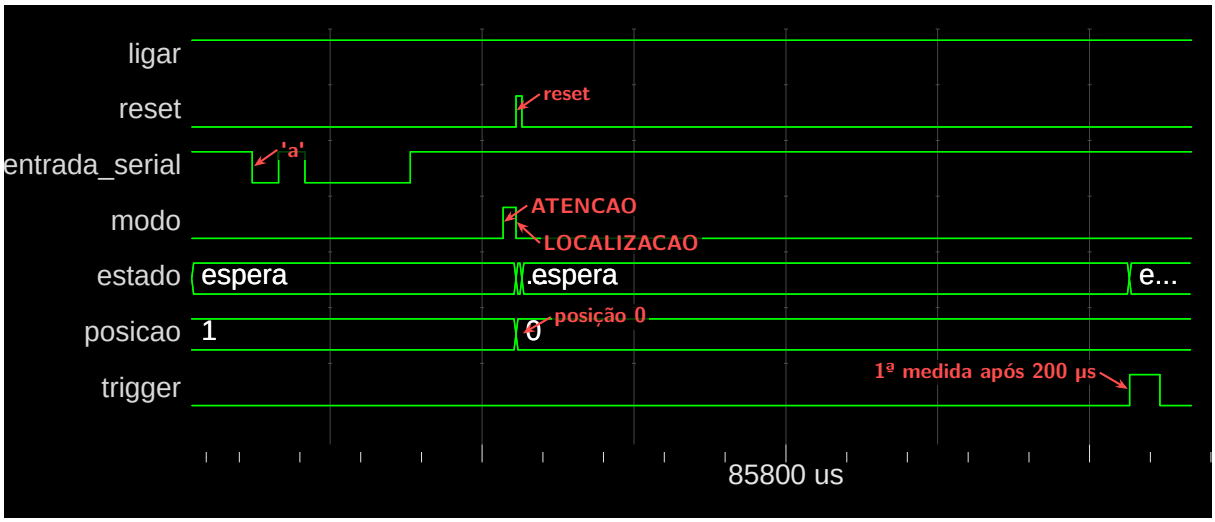


Figura 15: *Reset* logo após um `'a'` (CT19): volta ao modo de localização e à posição 000.

Tabela 14: Medidas nas formas de onda do `sonar2_tb`.

Grandeza	Esperado	Medido	Figura
Atraso `trigger` → *echo* (CT2)	400 µs (modelo do sensor)	400,00 µs	6
Largura de *echo* para 17 cm (CT2)	17 × 2941 ciclos = 999,94 µs	999,94 µs	6
Quadro 7E1 (quadros colados, CT6)	10 bits × 434 ciclos = 86,8 µs	86,80 µs	9
Start bit → `pronto` (CT4, CT8, CT10)	9,5 bits ≈ 82,5 µs	82,56 µs	7, 11
`pronto` → mudança de `modo`	1 ciclo (20 ns)	20 ns	7
Timeout a partir do `trigger` (CT7)	`TIMEOUT` = 20 ms, menos 3 ciclos até o `trigger`	19,99994 ms	10
Mensagem após o *timeout* (CT7)	8 caracteres ≈ 764 µs	764,84 µs	10
`'a'` na transmissão: `recebeu_atencao` antes de `fim_posicao` (CT10)	positivo (comando vale no ciclo)	673,5 µs	13
Religar → primeiro `trigger` (CT18)	`ESPERA` (200 µs) + preparação	200,10 µs	14

1.7 Arquivo de simulação

O `exp6_modelsim_t1ba5.zip` contém 25 módulos, 13 *testbenches*, os scripts `simula.do` e `simula_todos.do` e o leiaute de ondas do sonar. Após extrair, `do simula_todos.do` executa a regressão; `do simula.do` abre o sistema com os sinais agrupados. O resultado registrado para os 13 testes é reproduzido na Listagem 5.

2 Atividade 2: Planejamento da Execução Experimental

O plano cobre montagem, ensaios incrementais, monitoramento e síntese (Tabela 15). As evidências de execução serão registradas na Atividade 3.

Tabela 15: Itens da Atividade 2 do roteiro e seções deste planejamento.

Item	Conteúdo pedido	Seção
e.1	Montagem e testes incrementais	2.1.1, 2.1.2
e.2	Diagramas de energização e conexões	2.1.1
e.3	Circuitos de teste intermediários	2.1.2
e.4	Validação de cada módulo	2.1.2
e.5	Testes do sistema e de suas partes	2.1.3
e.6	Sinais de depuração, ferramentas e configurações	2.1.4, 2.1.5
f	Sequência de atividades na bancada	2.2
g	Designação de pinos e sinais de depuração	2.3
h	Arquivo QAR do projeto	2.4

2.1 Plano de execução experimental

2.1.1 Montagem e energização

A montagem reutiliza o conjunto sensor–servomotor da Experiência 5 (Figura 17) e acrescenta o TXD do CP2102 à `entrada_serial`. As conexões estão na Figura 16 e na Tabela 17. Executar com a placa desligada até a conferência de alimentação:

1. **GND comum.** Ligar o GND da DE0-CV (pinos 12 e 30 dos conectores) ao GND do HC-SR04, do servomotor, do CP2102 e do Analog Discovery. Todos precisam da mesma referência.

2. **Alimentação.** Conectar sensor e servomotor conforme a Tabela 16. Conferir a tensão antes de conectar os sinais à FPGA.

3. **Sinais.** Conectar GPIOs, CP2102 e instrumentos conforme a Tabela 17. As pontas do Analog Discovery ficam em paralelo com os fios.

4. **Parte mecânica.** Conferir que o HC-SR04 gira livremente com o eixo de 20° a 160°, sem prender cabos, com a posição central (90°) apontando para a frente.

5. **Computador.** Ligar o CP2102 (USB) e identificar a porta (COM no Windows). No *sketch* do Processing, ajustar só a variável `porta`; o formato 115200/7E1 já está definido.

6. **Conferência.** Com multímetro e a placa desligada, conferir a ausência de curto entre alimentações e GND. Energizar com `SW1` = 0 e medir 3,3 V no VCC do sensor e 5 V no VCC do servomotor.

7. **Programação.** Programar a FPGA com o `.sof` (Seção 2.4), com `SW9` a `SW7` em 000.

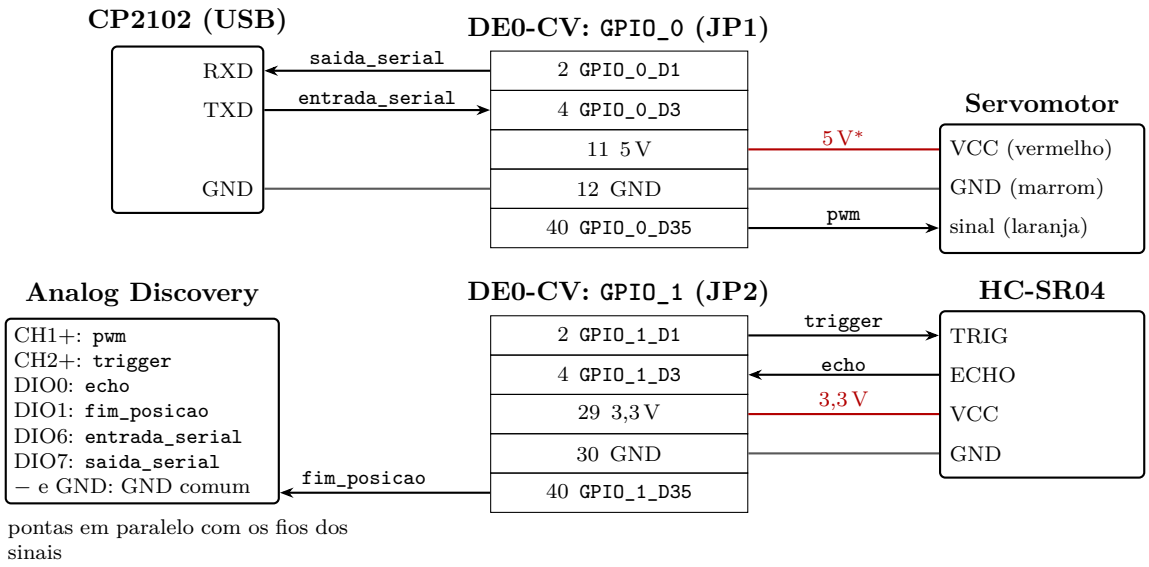


Figura 16: Ligações da montagem nos conectores de 40 pinos da DE0-CV. Em vermelho, alimentação; em cinza, GND (todos ligados entre si). *O servomotor pode ser alimentado pela fonte de 5 V da bancada, com o GND da fonte no GND comum.

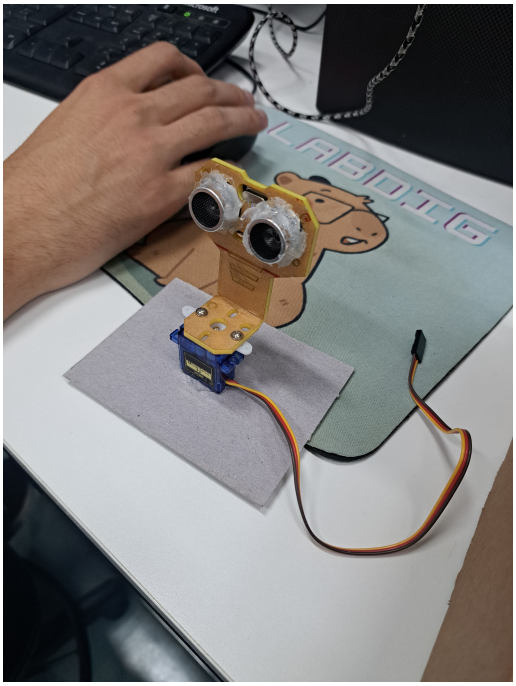


Figura 17: Conjunto do *kit* montado no desafio da Experiência 5: HC-SR04 sobre o eixo do servomotor.

Tabela 16: Alimentação de cada componente e cuidados.

Componente	Alimentação	Cuidados
DE0-CV	Fonte da própria placa.	Energizar com `SW1` = 0, depois de conferir a ausência de curto.
HC-SR04	3,3 V, pino 29 do `GPIO_1`.	Nunca em 5 V: o `echo` chegaria a 5 V à FPGA.
Servomotor	5 V: pino 11 do `GPIO_0` ou fonte da bancada.	Nunca por pino de E/S. GND da fonte no GND comum. Picos de corrente podem derrubar a tensão (Tabela 21).
CP2102	USB do computador.	GND no GND da FPGA; TXD em `entrada_serial` e RXD em `saida_serial` (cruzados).
Analog Discovery	USB do computador.	Pontas − e GND no GND comum.

Tabela 17: Ligações da montagem (conectores de 40 pinos da DE0-CV).

Origem (DE0-CV)	Pino	Destino	Sinal
`GPIO_0_D1`	2	RXD do CP2102; DIO7	`saida_serial`
`GPIO_0_D3`	4	TXD do CP2102; DIO6	`entrada_serial`
`GPIO_0` 5 V	11	VCC do servomotor (ou fonte da bancada)	alimentação
`GPIO_0` GND	12	GND do servomotor e do CP2102	referência
`GPIO_0_D35`	40	Sinal do servomotor; Scope CH1+	`pwm`
`GPIO_1_D1`	2	TRIG do HC-SR04; Scope CH2+	`trigger`
`GPIO_1_D3`	4	ECHO do HC-SR04; DIO0	`echo`
`GPIO_1` 3,3 V	29	VCC do HC-SR04	alimentação
`GPIO_1` GND	30	GND do HC-SR04 e do Analog Discovery	referência
`GPIO_1_D35`	40	DIO1 do Analog Discovery	`fim_posicao`

2.1.2 Circuitos de teste intermediários e validação dos módulos

Executar EU1 a EU4 antes dos ensaios de integração EI1 a EI3 (Tabela 18). O circuito utilizado é o próprio `sonar2`, observado pelas páginas de depuração. Com `ligar` = 0, a UC não inicia novos ciclos, mas o receptor e o PWM continuam ativos. Os diagnósticos estão concentrados na Tabela 21.

Tabela 18: Testes das etapas intermediárias.

Teste	Preparação e procedimento	Critério de aprovação
EU1	Unir TXD e RXD do CP2102, sem FPGA; enviar `'a'`, `'v'` e `'x'` em 115200/7E1.	Receber os três caracteres sem alteração.
EU2	`ligar` = 0; enviar os mesmos caracteres. Observar páginas 001 e 011 e UART em DIO6.	HEX1–HEX0: 61, 76 e 78; paridade = 1; LEDR9 apagado.
EU3	Localização; Scope CH1 no PWM. Percorrer as oito posições indicadas por HEX3.	Período de 20 ms; larguras da Tabela 4; uma largura por posição.
EU4	Página 000; objeto a 10 e 30 cm, medidos com régua. Registrar três leituras em cada distância; observar `trigger` e `echo`.	Pulso de `trigger` de 10 µs; distância dentro da faixa de ensaio da Seção 2.1.3.
EI1	Localização; Processing recebendo. Observar uma varredura completa.	Mensagens `aaa,ddd#`, ângulos de 020 a 160 e retorno a 020.
EI2	Objeto fixo; aplicar `'a'` e depois `'v'`; observar posição, LEDR9 e mensagens.	Em atenção, ângulo fixo e mensagens contínuas; ao voltar, avanço a partir da posição mantida.
EI3	Executar ET1 a ET12 com o sistema completo.	Aprovação pelos critérios da Tabela 19.

2.1.3 Testes do sistema completo

Os ensaios ET1 a ET12 seguem a Tabela 19. Para medir distância, alinhar um objeto plano ao sensor e coletar três leituras por posição. Adota-se uma faixa de ±2 cm em torno dos alvos de 10 e 30 cm como **critério deste ensaio**, sem atribuir essa tolerância à precisão nominal do sensor. Registrar a distância da régua e as leituras obtidas.

Enviar os comandos pela janela do Processing ou pelo terminal, usando apenas um programa por vez. Registrar a configuração, os estímulos e os resultados na Atividade 3.

Tabela 19: Casos de teste na bancada.

Teste	Preparação e procedimento	Critério de aprovação
ET1	Acionar o *reset*, soltá-lo e colocar `ligar` = 1.	Modo 0; primeira mensagem com ângulo 020; varredura a partir de 000.
ET2	Em localização, observar em HEX3 e no Processing os oito ciclos seguintes à primeira posição.	Sequência 000 a 111, seguida de 000; ângulos de 020 a 160, seguidos de 020.
ET3	Enviar `'a'` durante a espera; observar três ciclos seguintes.	LEDR9 aceso; posição e ângulo mantidos nas três mensagens.

Continuação da tabela.

Teste	Preparação e procedimento	Critério de aprovação
ET4	Em atenção, posicionar o objeto a 10 e 30 cm; coletar três mensagens por alvo.	Ângulo fixo; distâncias entre 8 e 12 cm e entre 28 e 32 cm, respectivamente.
ET5	Em atenção, enviar `'v'`; observar dois ciclos seguintes.	LEDR9 apagado; varredura retomada a partir da posição mantida.
ET6	Em atenção, enviar `'A'`, `'x'`, `'V'` e espaço em 7E1; depois, `'v'` em 8N1.	Modo e posição mantidos; o quadro de `'v'` em 8N1 apresenta paridade incorreta.
ET7	Enviar `av` e, em outro ciclo, `va`, antes da bifurcação. Repetir durante o envio da mensagem.	O último comando determina o modo: localização para `av`; atenção para `va`.
ET8	Em atenção, desligar e religar por SW1.	Modo 0; primeira mensagem após religar com ângulo 020.
ET9	Em atenção, acionar e soltar SW0.	Modo 0; posição reiniciada em 000.
ET10	Em atenção, desconectar a saída ECHO do sensor e manter a entrada da FPGA em 0.	Espera de 10 s pela medida; mensagem `aaa,000#`; posição mantida.
ET11	Medir `pwm`, `trigger`, mensagens e `fim_posicao` nos dois modos, conforme a Tabela 20.	Pausa de 2 s acrescida da medida e do envio; um pulso de `fim_posicao` de 20 ns por ciclo.
ET12	Selecionar páginas 000 a 011 após uma medida concluída.	Conferir estado, posição, distância, caracteres e modo com os sinais da Tabela 8.

2.1.4 Sinais de depuração e monitoramento

A Tabela 20 define os canais e as configurações. As conexões físicas estão na Tabela 17; os displays e LEDs, nas Tabelas 8 e 9.

Tabela 20: Monitoramento dos sinais na bancada.

Sinal / canal	Instrumento e configuração	Grandeza a verificar
RX / DIO6	Protocol UART: 115200, 7 bits, paridade par, 1 stop; 20 µs/div.	Quadro de 86,8 µs; `'a'` = 61H e `'v'` = 76H.
TX / DIO7	Protocol UART: mesma configuração; 100 µs/div.	Oito caracteres, `aaa,ddd#`, em cerca de 765 µs.
`pwm` / CH1+	Scope: disparo na subida a 1,5 V; 500 µs/div para largura; 5 ms/div para período.	Período de 20 ms; larguras da Tabela 4.

Continuação da tabela.

Sinal / canal	Instrumento e configuração	Grandeza a verificar
`trigger` / CH2+	Scope: captura única, subida; 5 µs/div.	Pulso de 10 µs.
`echo` / DIO0	Analisador lógico: subida; 2 ms/div.	Largura aproximada de 58,8 µs por cm.
`fim_posicao` / DIO1	Analisador lógico: 100 MHz para largura; modo Record para intervalo entre ciclos.	Pulso de 20 ns; intervalo composto por espera, medida e envio.

Processing. Selecionar a porta do CP2102, clicar na janela do radar e teclar para enviar comandos. Capturar a interface gráfica e o console com as mensagens recebidas. Fechar o terminal antes de abrir a mesma porta no Processing.

2.1.5 Estratégia de depuração

Investigar na ordem: alimentação e conexões; chaves e páginas de depuração; sinais no Analog Discovery; comparação com a simulação. A Tabela 21 relaciona sintomas a verificações específicas.

Tabela 21: Diagnóstico de falhas na bancada.

Sintoma	Causa provável	Ação
Nada acontece, HEX5 = 0	`SW1` = 0 ou `SW0` = 1.	Ajustar as chaves.
`'a'` sem efeito; página 011 com 61 e paridade ok = 0	Terminal fora de 7E1.	Configurar 115200/7E1.
`'a'` sem efeito; página 001 não muda; LEDR5 não pisca	Fio TXD, pino ou GND.	Repetir EU1 e EU2; conferir `GPIO_0_D3`.
LEDR9 acende, mas o servomotor anda mais uma posição	`'a'` chegou depois da bifurcação daquele ciclo.	Comportamento previsto: para no ciclo seguinte.
Comando ignorado com o sonar desligado	Previsto: desligado, o modo fica em localização.	Ligar antes de enviar.
HEX5 = 4 e HEX4 = 3, `aaa,000#`	Sensor sem resposta (*timeout*).	Checar VCC, GND e ECHO.
HEX5 = 6 preso	Falha na transmissão.	Analisar `saida_serial`.
Servomotor inerte ou tremendo	Queda de tensão do servomotor.	Verificar os 5 V e o GND da fonte.

Continuação da tabela.

Sintoma	Causa provável	Ação
Erro no terminal ou no gráfico	Porta ocupada ou configuração errada.	Fechar o outro programa; conferir 115200/7E1.

2.2 Sequência de atividades na bancada

Executar a sequência abaixo, avançando após a aprovação de cada etapa:

1. Montagem com o *kit* e conferência das alimentações (Seção 2.1.1).
2. Testes de unidade EU1 a EU4: CP2102, recepção serial com o sonar desligado, servomotor e sensor do *kit* (Tabela 18).
3. Testes de integração EI1 (sonar em localização, como na Experiência 5) e EI2 (comandos com objeto fixo).
4. Ensaios ET1 a ET12, com registro de formas de onda, displays e saída do Processing na Atividade 3.
5. Demonstração ao professor: varredura, `'a'`, medidas com o servomotor parado, `'v'` e caracteres ignorados. Envio do QAR final (`exp6_final_t1ba5.qar`).
6. Desafio (Seção 4).

2.3 Designação de pinos

A Tabela 22 segue a designação mínima da Tabela 2 do roteiro, completada com os pinos da FPGA, e a Tabela 23 traz os sinais de depuração do grupo. Todas as portas usam o padrão 3.3-V LVTTL.

Tabela 22: Designação de pinos (Tabela 2 do roteiro).

Sinal	Recurso da DE0-CV	Pino da FPGA
`clock`	`CLOCK_50`	M9
`reset`	chave `SW0`	U13
`ligar`	chave `SW1`	V13
`entrada_serial`	`GPIO_0_D3`	C16
`trigger`	`GPIO_1_D1`	A12
`echo`	`GPIO_1_D3`	B12
`pwm`	`GPIO_0_D35`	T15
`saida_serial`	`GPIO_0_D1`	B16
`fim_posicao`	`GPIO_1_D35`	K16

Tabela 23: Pinos dos sinais de depuração.

Sinal	Recurso da DE0-CV	Pinos da FPGA
`sel_depuracao[0..2]`	chaves `SW7`, `SW8`, `SW9`	AA13, AB13, AB12
`db_ligar`	LEDR0	AA2
`db_echo`	LEDR1	AA1
`db_trigger`	LEDR2	W2
`db_pwm`	LEDR3	Y3
`db_saida_serial`	LEDR4	N2
`db_entrada_serial`	LEDR5	N1
`db_posicao[0]`	LEDR6	U2
`db_posicao[1]`	LEDR7	U1
`db_posicao[2]`	LEDR8	L2
`db_modo`	LEDR9	L1
`hex0[0..6]`	HEX0	U21, V21, W22, W21, Y22, Y21, AA22
`hex1[0..6]`	HEX1	AA20, AB20, AA19, AA18, AB18, AA17, U22
`hex2[0..6]`	HEX2	Y19, AB17, AA10, Y14, V14, AB22, AB21
`hex3[0..6]`	HEX3	Y16, W16, Y17, V16, U17, V18, V19
`hex4[0..6]`	HEX4	U20, Y20, V20, U16, U15, Y15, P9
`hex5[0..6]`	HEX5	N9, M8, T14, P14, C1, C2, W19

Nas listas de pinos de `sel_depuracao` e `hex0` a `hex5`, a ordem corresponde aos índices crescentes do sinal. As chaves são ativas em nível alto; os displays, em nível baixo.

2.4 Compilação e arquivo QAR

O circuito de teste da experiência é o próprio `sonar2` (Figura 18), compilado com os valores reais de `ESPERA` e `TIMEOUT` e os pinos das Tabelas 22 e 23 (Figura 19). O arquivo de restrições `de0_cv.sdc` da Experiência 5 foi estendido às novas portas (`entrada_serial`, `sel_depuracao`, displays e LEDs), com `set_false_path` nas entradas e saídas assíncronas; o Timing Analyzer não aponta caminhos sem restrição. A compilação termina sem erros e com *slack* positivo em todos os cantos de temporização (Tabela 24).

Tabela 24: Resultado da compilação no Quartus Prime 25.1 Lite.

Item	Resultado
Lógica (ALMs)	258 de 18 480 (1%)
Registradores	319
Pinos	64 de 224 (29%)
Memória, DSP e PLL	não usados
Pior *slack* de *setup*	13,737 ns (Slow 1100 mV 85 °C)
Pior *slack* de *hold*	0,168 ns (Fast 1100 mV 0 °C)
Pior *slack* de *recovery*/*removal*	16,387 ns / 0,516 ns
$F_{\max}$	159,67 MHz (Slow 85 °C), para 50 MHz exigidos

Foram registrados seis avisos: três sobre a inicialização da `rom_angulos_8x24` (10030); um sobre a falta de especificação de corrente e *slew rate* das saídas (15714); um sobre a distribuição do `reset` em rede global a partir de um pino sem *clock* dedicado (16406); e um sobre a licença Lite do Quartus (292013). O projeto compilado é entregue no arquivo `exp6_t1ba5.qar`, junto com este planejamento. Para conferir que o arquivo está completo, ele foi restaurado em outra pasta e compilado de novo, com o mesmo resultado.

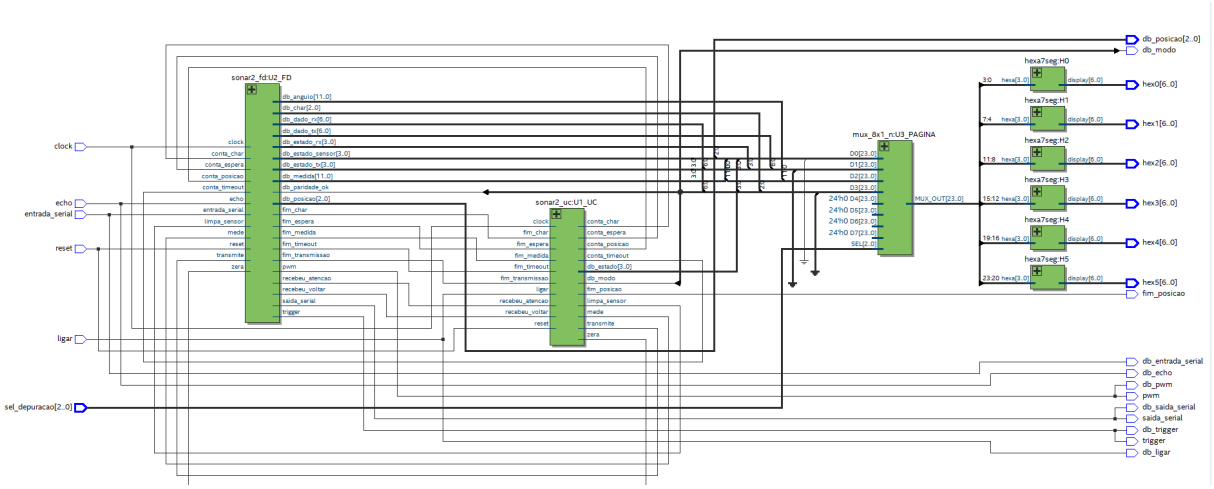
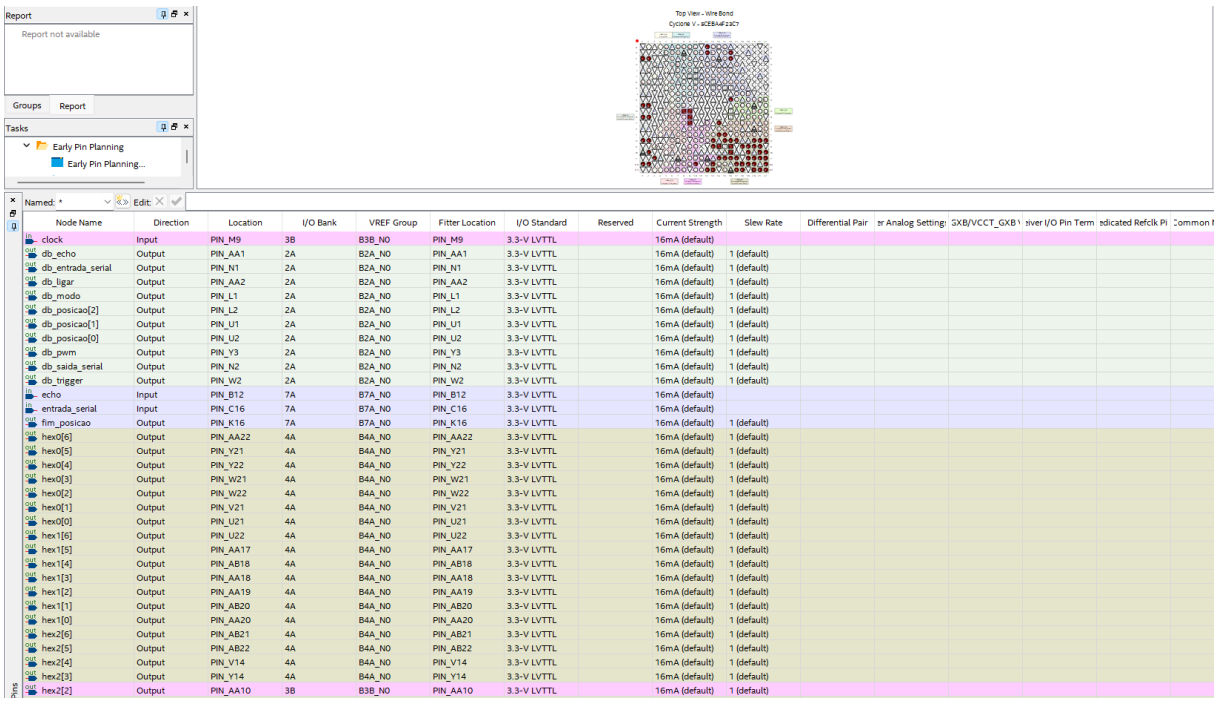


Figura 18: Diagrama RTL do módulo `sonar2` (UC, FD, multiplexador de páginas e decodificadores).



Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair
clock	Input	PIN_M9	3B	B3B_N0	PIN_M9	3.3-V LVTTL		16mA (default)		
db_echo	Output	PIN_AA1	2A	B2A_N0	PIN_AA1	3.3-V LVTTL		16mA (default)	1 (default)	
db_entrada_serial	Output	PIN_N1	2A	B2A_N0	PIN_N1	3.3-V LVTTL		16mA (default)	1 (default)	
db_ligar	Output	PIN_AA2	2A	B2A_N0	PIN_AA2	3.3-V LVTTL		16mA (default)	1 (default)	
db_modo	Output	PIN_L1	2A	B2A_N0	PIN_L1	3.3-V LVTTL		16mA (default)	1 (default)	
db_posicao[2]	Output	PIN_L2	2A	B2A_N0	PIN_L2	3.3-V LVTTL		16mA (default)	1 (default)	
db_posicao[1]	Output	PIN_U1	2A	B2A_N0	PIN_U1	3.3-V LVTTL		16mA (default)	1 (default)	
db_posicao[0]	Output	PIN_U2	2A	B2A_N0	PIN_U2	3.3-V LVTTL		16mA (default)	1 (default)	
db_pwm	Output	PIN_Y3	2A	B2A_N0	PIN_Y3	3.3-V LVTTL		16mA (default)	1 (default)	
db_saida_serial	Output	PIN_N2	2A	B2A_N0	PIN_N2	3.3-V LVTTL		16mA (default)	1 (default)	
db_trigger	Output	PIN_W2	2A	B2A_N0	PIN_W2	3.3-V LVTTL		16mA (default)	1 (default)	
echo	Input	PIN_B12	7A	B7A_N0	PIN_B12	3.3-V LVTTL		16mA (default)		
entrada_serial	Input	PIN_C16	7A	B7A_N0	PIN_C16	3.3-V LVTTL		16mA (default)		
fim_posicao	Output	PIN_K16	7A	B7A_N0	PIN_K16	3.3-V LVTTL		16mA (default)	1 (default)	
hex0[6]	Output	PIN_AA22	4A	B4A_N0	PIN_AA22	3.3-V LVTTL		16mA (default)	1 (default)	
hex0[5]	Output	PIN_Y21	4A	B4A_N0	PIN_Y21	3.3-V LVTTL		16mA (default)	1 (default)	
hex0[4]	Output	PIN_Y22	4A	B4A_N0	PIN_Y22	3.3-V LVTTL		16mA (default)	1 (default)	
hex0[3]	Output	PIN_W21	4A	B4A_N0	PIN_W21	3.3-V LVTTL		16mA (default)	1 (default)	
hex0[2]	Output	PIN_W22	4A	B4A_N0	PIN_W22	3.3-V LVTTL		16mA (default)	1 (default)	
hex0[1]	Output	PIN_V21	4A	B4A_N0	PIN_V21	3.3-V LVTTL		16mA (default)	1 (default)	
hex0[0]	Output	PIN_U21	4A	B4A_N0	PIN_U21	3.3-V LVTTL		16mA (default)	1 (default)	
hex1[6]	Output	PIN_U22	4A	B4A_N0	PIN_U22	3.3-V LVTTL		16mA (default)	1 (default)	
hex1[5]	Output	PIN_AA17	4A	B4A_N0	PIN_AA17	3.3-V LVTTL		16mA (default)	1 (default)	
hex1[4]	Output	PIN_AB18	4A	B4A_N0	PIN_AB18	3.3-V LVTTL		16mA (default)	1 (default)	
hex1[3]	Output	PIN_AA18	4A	B4A_N0	PIN_AA18	3.3-V LVTTL		16mA (default)	1 (default)	
hex1[2]	Output	PIN_AA19	4A	B4A_N0	PIN_AA19	3.3-V LVTTL		16mA (default)	1 (default)	
hex1[1]	Output	PIN_AB20	4A	B4A_N0	PIN_AB20	3.3-V LVTTL		16mA (default)	1 (default)	
hex1[0]	Output	PIN_AA20	4A	B4A_N0	PIN_AA20	3.3-V LVTTL		16mA (default)	1 (default)	
hex2[6]	Output	PIN_AB21	4A	B4A_N0	PIN_AB21	3.3-V LVTTL		16mA (default)	1 (default)	
hex2[5]	Output	PIN_AB22	4A	B4A_N0	PIN_AB22	3.3-V LVTTL		16mA (default)	1 (default)	
hex2[4]	Output	PIN_V14	4A	B4A_N0	PIN_V14	3.3-V LVTTL		16mA (default)	1 (default)	
hex2[3]	Output	PIN_Y14	4A	B4A_N0	PIN_Y14	3.3-V LVTTL		16mA (default)	1 (default)	
hex2[2]	Output	PIN_AA10	3B	B3B_N0	PIN_AA10	3.3-V LVTTL		16mA (default)	1 (default)	

Figura 19: *Pin Planner para o projeto sonar2.*

3 Atividade 3: Procedimento Experimental na Bancada

Registro reservado para a execução na bancada. Os campos em vermelho ainda não representam resultados experimentais. Preenchê-los com as evidências dos ensaios da Atividade 2; registrar desvios e soluções na Seção 3.3. A demonstração e o QAR final são entregues à parte.

3.1 Etapas intermediárias

Testes EU1 a EU4, EI1 e EI2 (Tabela 18).

[Inserir: terminal em 115200/7E1 com TXD e RXD do CP2102 unidos, mostrando os caracteres digitados e devolvidos]

Figura 20: EU1: CP2102 em laço (*loopback*), sem FPGA.

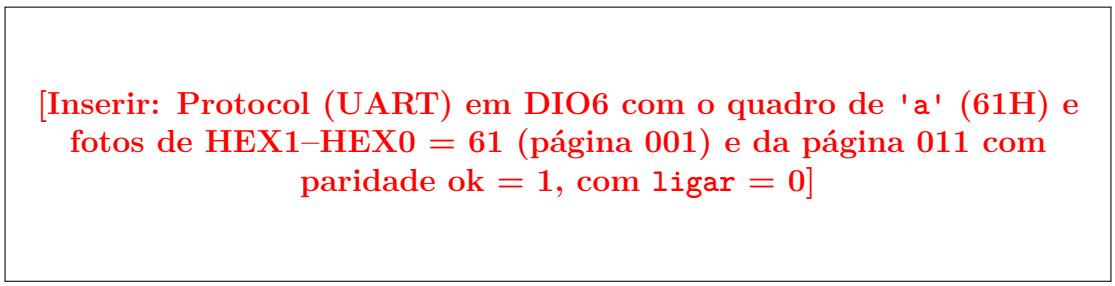


Figura 21: EU2: recepção serial com o sonar desligado.

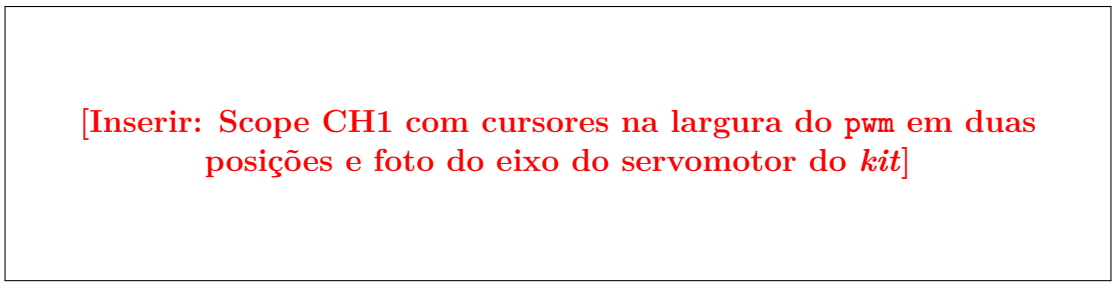


Figura 22: EU3: servomotor do *kit*.

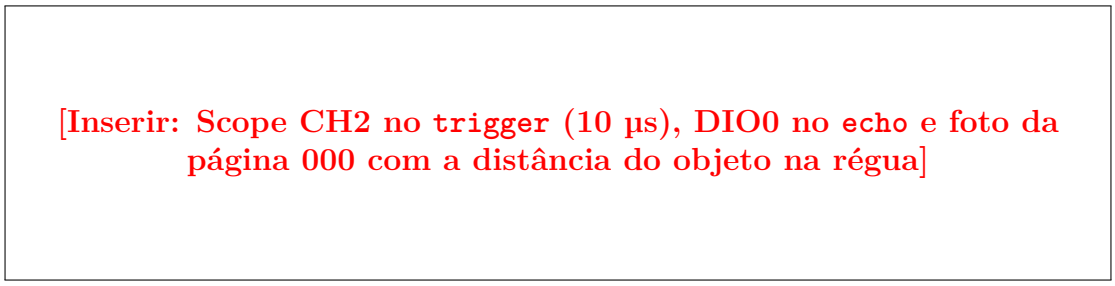


Figura 23: EU4: sensor do *kit*.



Figura 24: EI1: sonar no modo de localização.

[Inserir: fotos de LEDR9 e LEDR8–LEDR6 antes e depois de `'a'` e de `'v'`; terminal com as mensagens no mesmo ângulo]

Figura 25: EI2: comandos com objeto fixo.

3.2 Sistema completo

Testes ET1 a ET12 com o sensor e o servomotor do *kit* (Tabela 19).

Tabela 25: Resultados dos testes na bancada.

Teste	Descrição	Observado	Result.
ET1	*Reset* e ligar	[a preencher]	
ET2	Varredura	[a preencher]	
ET3	Comando `'a'`	[a preencher]	
ET4	Medidas em atenção	[a preencher]	
ET5	Comando `'v'`	[a preencher]	
ET6	Caracteres ignorados	[a preencher]	
ET7	Comandos seguidos	[a preencher]	
ET8	Religar em atenção	[a preencher]	
ET9	*Reset* em atenção	[a preencher]	
ET10	Sem *echo* em atenção	[a preencher]	
ET11	Período	[a preencher]	
ET12	Páginas de depuração	[a preencher]	

[Inserir: Protocol (UART) com DIO6 (`'a'`) e DIO7 (mensagens): o ângulo das mensagens para de mudar depois do `'a'`]

Figura 26: ET3: comando `'a'` e mensagens seguintes.

[Inserir: tela do Processing no modo de atenção (mesmo ângulo, distâncias variando) e console com “Enviando tecla ‘a’ ” e as mensagens recebidas]

Figura 27: ET4: medidas no modo de atenção no Processing.

[Inserir: fotos das páginas 000, 001, 010 e 011 dos displays e dos LEDs]

Figura 28: ET12: páginas de depuração.

3.3 Ocorrências experimentais

Registro de desvios em relação ao plano da Seção 2 e das soluções adotadas na bancada:

- **Montagem e *kit*.** [a preencher]
- **Comunicação serial.** [a preencher]
- **Sensor e servomotor.** [a preencher]
- **Outras observações.** [a preencher]

4 Atividade 4: Desafio

[Seção a completar.] A atividade adicional será apresentada pelo professor na aula (item o do roteiro). Plano para documentá-la:

- Especificação do desafio: **[a preencher]**.
- Alterações no projeto: novos estados ou ramos da UC, componentes e sinais do FD, sinais de depuração (páginas livres 100 a 111 dos displays): **[a preencher]**.
- Verificação: casos acrescentados ao `sonar2_tb` e testes na bancada: **[a preencher]**.
- Resultados e arquivos entregues: **[a preencher]**.